Thank you for purchasing a license for XFur Studio™ 4. This offline version of the documentation is provided for your convenience. However, you are heavily encouraged to read the online version instead for more up-to-date information about the product, its features and full capabilities. If you require assistance or support to use our software, please contact us at support@irreverent-software.com with the following information:

- Invoice number / order ID / proof of purchase from the Unity Asset Store
- Unity version
- Rendering pipeline used, and its version
- Asset version used
- Graphics API used
- Any relevant information about the issue

We will get back to you as soon as possible. We usually reply to all support requests within two to four business days.

Index

Differences between XFur Studio™ 4 Editions

Features	XFur Studio™ 4 Core Edition	XFur Studio™ 4 URP Edition	XFur Studio™ 4 Personal / Team Edition
GPU Accelerated Fur (with Vertex Buffers)	✓	✓	✓
Universal Rendering Pipeline Support	✓	✓	✓
High Definition Rendering Pipeline Support	✗	✗	✓
Legacy (Built-in) Rendering Pipeline Support	✓	✗	✓
Advanced Fur Coloring and Tint variation	✓	✓	✓
Fur styling + grooming within the Unity Editor	✓	✓	✓
Fur styling + grooming at runtime through the XFur Studio™ API	✓	✓	✓
Fur styling at runtime with Painter objects	✓	✓	✓
Bake fur styling and grooming to mesh	✗	✓	✓
Motion Vectors for URP (Unity 6+)	✗	✓	✓
Motion Vectors for HDRP (Unity 2021+)	✗	✗	✓
Emissive Fur Rendering	✗	✓	✓
Curly Fur Rendering	✗	✓	✓
GPU accelerated Physics	✗	✓	✓
GPU accelerated weather sim	✗	✓	✓
UV based decals	✗	✓	✓
Touch bending spheres support	✗	✓	✓
Dynamic masking through textures	✗	✓	✓
Dynamic LOD & LOD component support	✗	✓	✓
Spawn-time Randomization support	✗	✓	✓
Custom modules support	✗	✓	✓
Source code access	✓	✓	✓
Tiger demo scene	✗	✓	✓

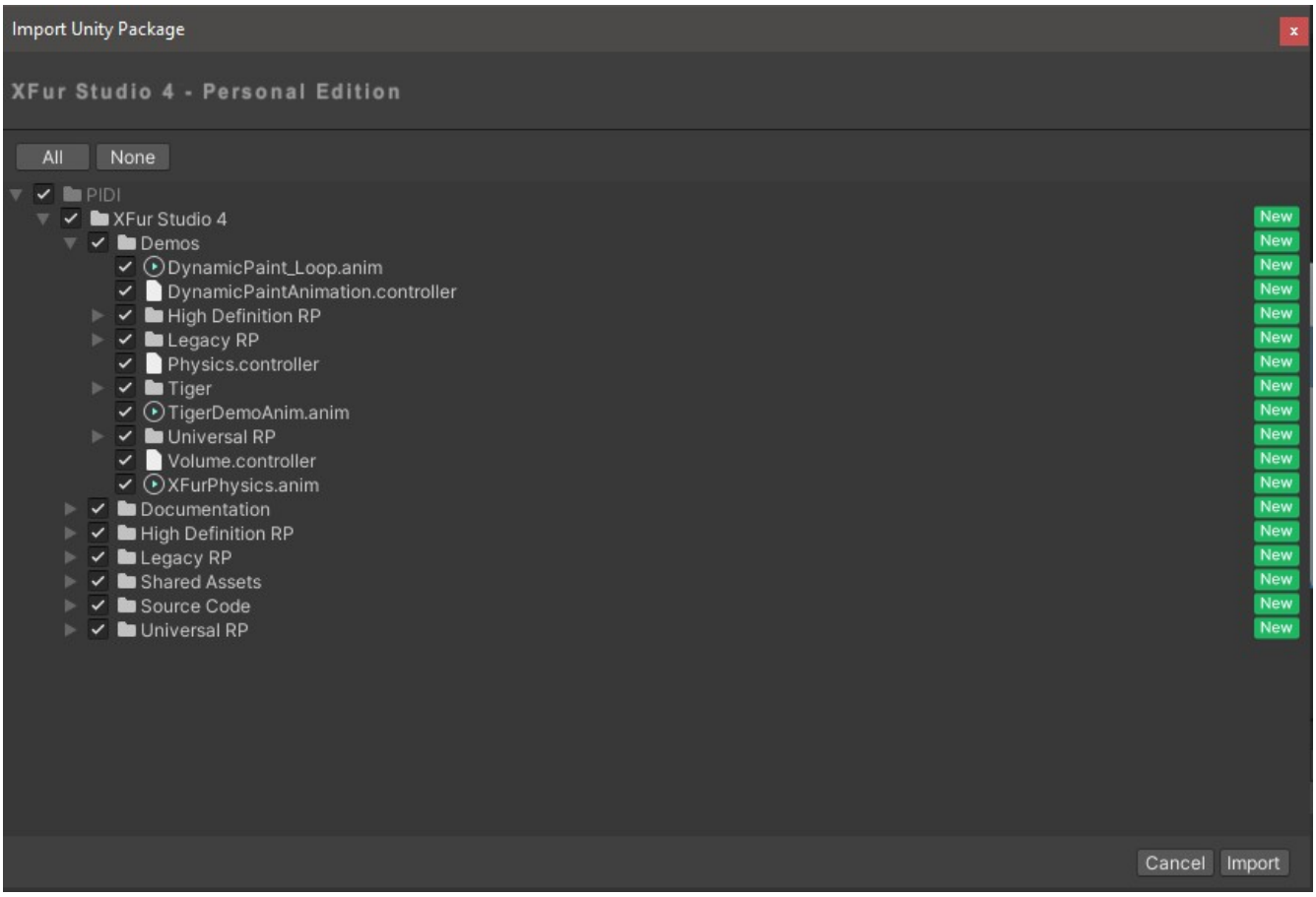
Installation

Installing XFur Studio™ into your project is fairly easy, but there are a few considerations to take into account depending on the exact edition you are using. XFur Studio™ 4 comes in 4 different editions:

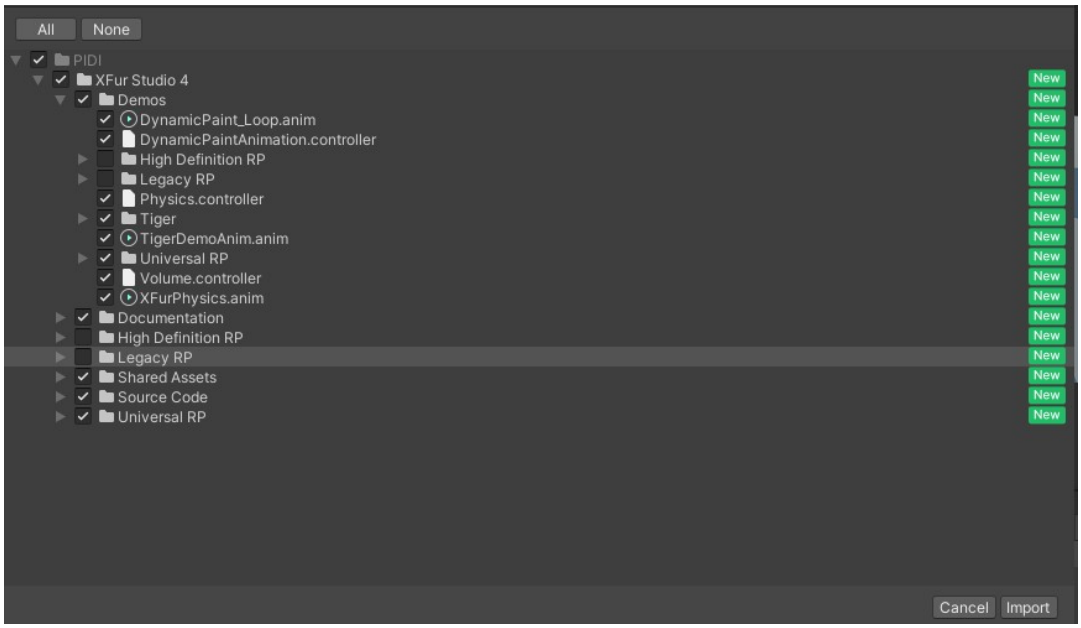
- **XFur Studio™ 4 – Personal Edition**
- **XFur Studio™ 4 – Team Edition**
- **XFur Studio™ 4 – URP Edition**
- **XFur Studio™ 4 – Core Edition**

The Personal and Team Edition are the same content-wise. The URP Edition is a fully featured version of the software but limited to URP support, while the Core Edition contains only the most basic features for both URP and the Legacy / Built-in rendering pipelines.

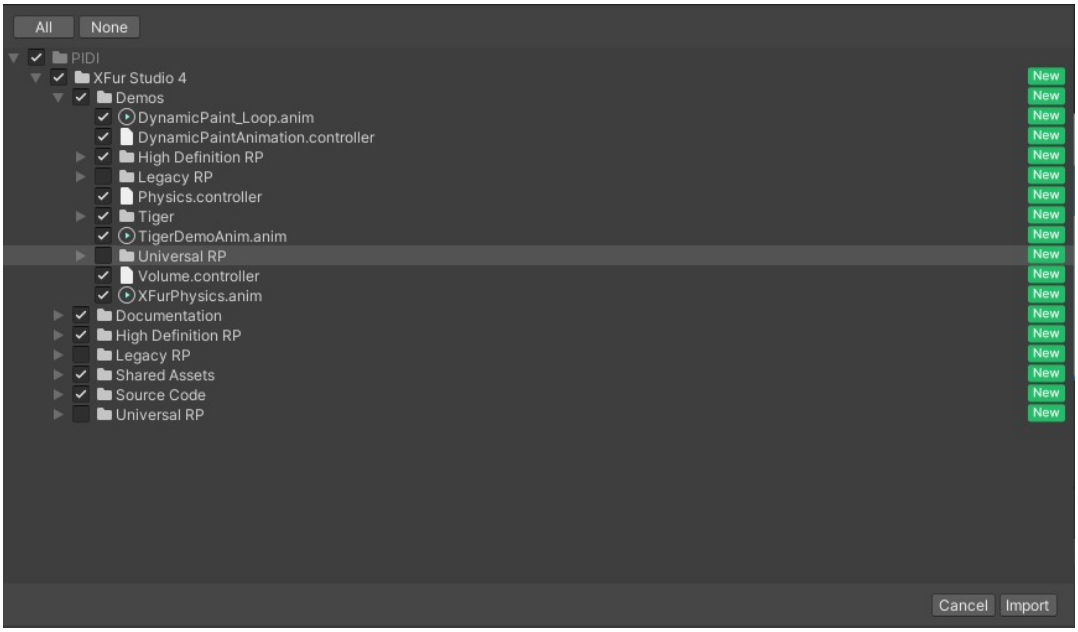
For all editions, you can Download the asset directly from the Package Manager. Once downloaded, you can import it into your project and a dialog box showing all the content available for import will be displayed.



When installing the Personal, Team or Core Editions you must deselect any folders that do not belong to the rendering pipeline you are using for your project. This means that for URP projects, for example, you must deselect the “Legacy RP” folder within the “Demos” folder, as well as the “Legacy RP” folder within the root folder of the asset. For Legacy RP projects, the “Universal RP” and “High Definition RP” folders within the root and “Demos” folders must be deselected instead.



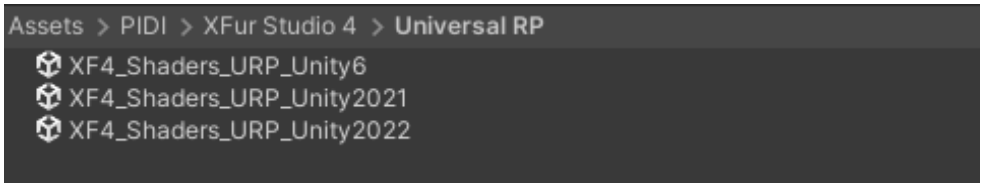
Example of content selection for the Universal Rendering Pipeline



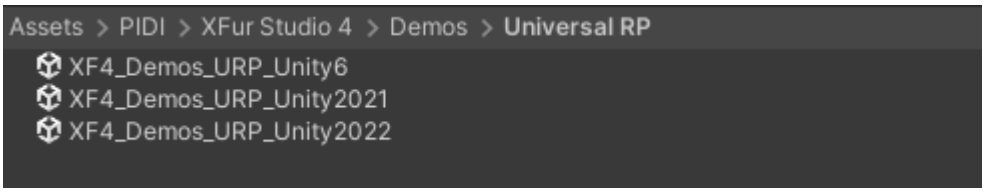
Example of content selection for the High Definition Rendering Pipeline

When using the URP edition you can safely import all the contents of the asset.

Once all of the content of the asset has been imported into your project, if you are using the Core, Personal or Team Editions of the asset, you will need to unpack the unitypackage file that matches your Unity version more closely from within the folder that matches your current rendering pipeline (if you are using the High Definition or Universal Rendering Pipelines). You must do this for the Demos as well.



Example of the different Shader unity packages for different Unity versions with Universal RP



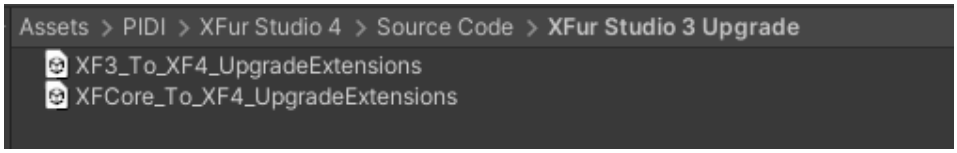
Example of the different Demo unity packages for different Unity versions with Universal RP

Upgrading from XFur Studio™ 3 to XFur Studio™ 4

Upgrading an XFur Studio Instance from version 3 to version 4 is a very easy process. **Do not remove your installation of XFur Studio™ 3 / XFur Studio™ Core until all of your models and content have been upgraded successfully.**

First, unpack one of the unity package files inside of the “XFur Studio 4/Source Code/XFur Studio 3 Upgrade” folder. These packages add the necessary code bridges that will make the upgrade process smooth and painless.

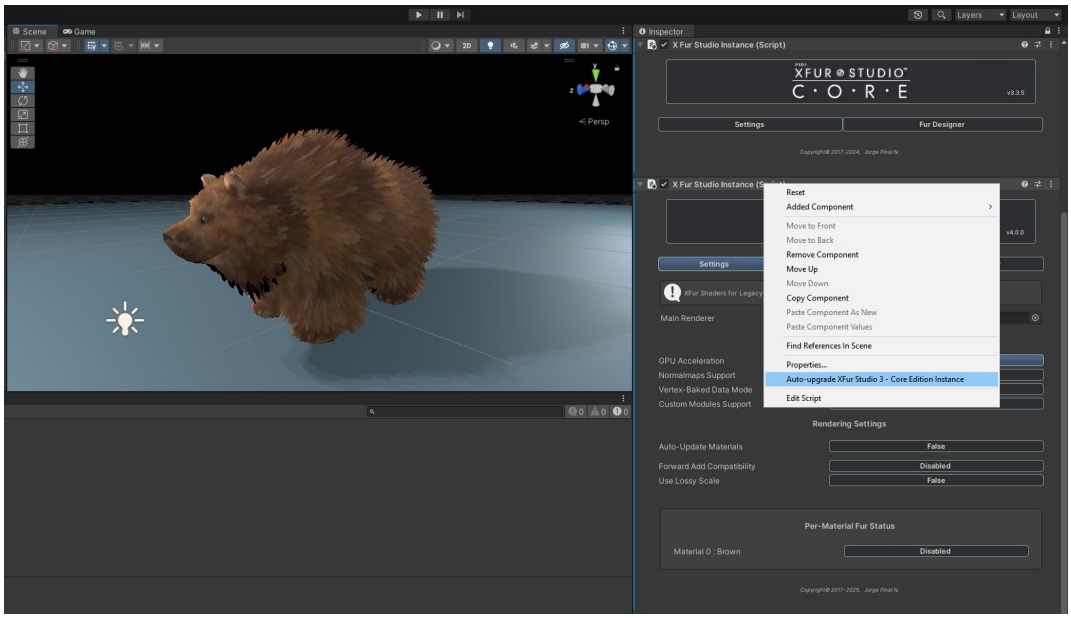
Important Notice: If you are upgrading from XFur Studio™ Core to XFur Studio™ 4, please make sure to unpack the “***XFCore_To_XF4_UpgradeExtensions***” file. If you are upgrading from XFur Studio™ 3 Personal, Team or URP Editions, unpack the “***XF3_To_XF4_UpgradeExtensions***” file instead.



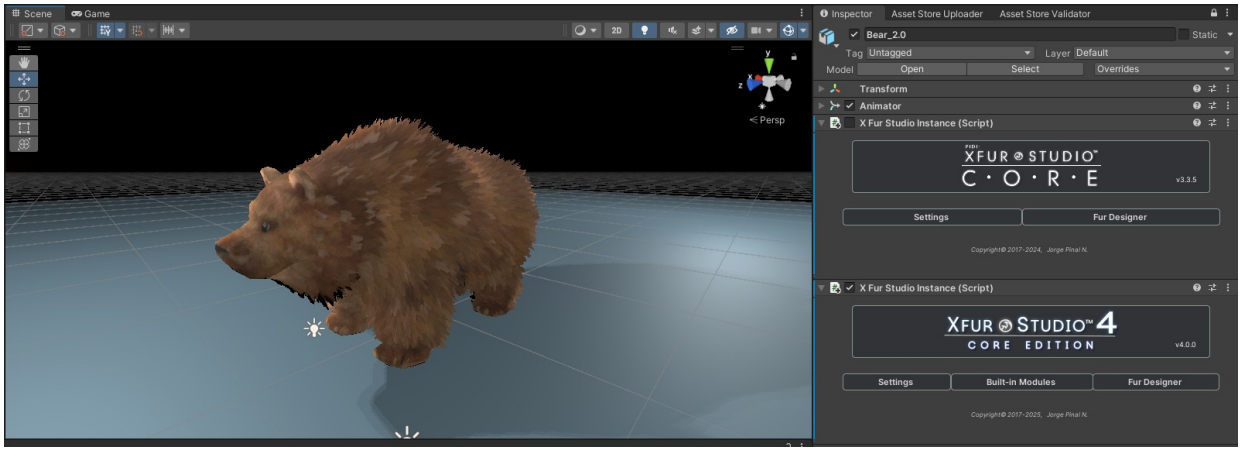
The Upgrade Extensions package. Unpack its contents to install all necessary upgrade utilities.

Once the package has been successfully unpacked and its contents imported into your project, you are ready to begin the upgrade process.

On each model using XFur Studio™ 3, add the corresponding XFur Studio™ 4 Instance component as well. Then, right click on the XFur Studio™ 4 Instance component and click on the “**Auto-Upgrade XFur Studio™ 3 / Core Instance**” option.



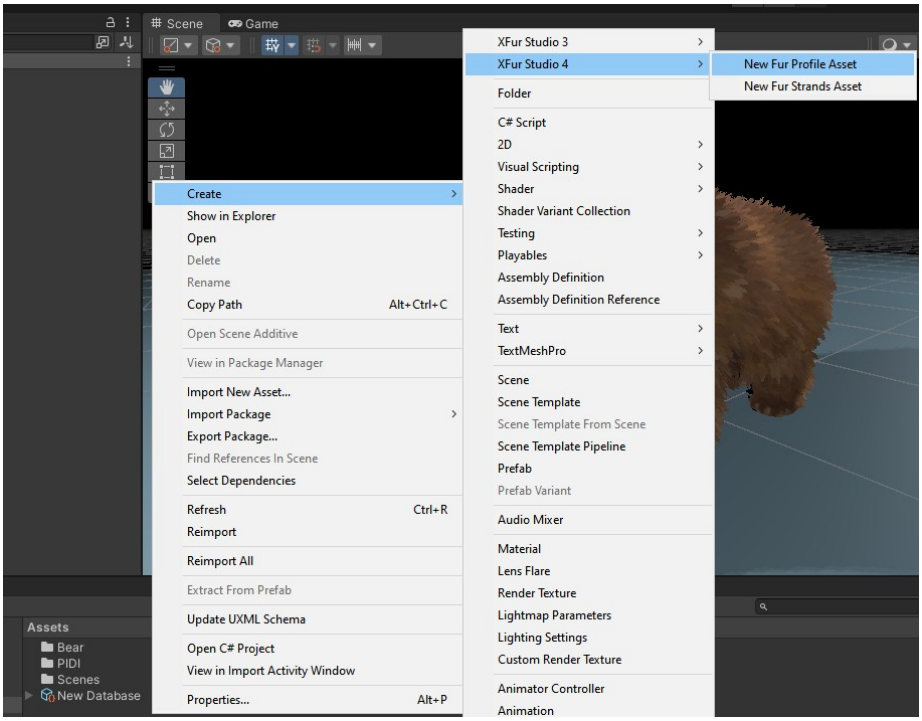
Once the process is done, the old version 3 instance will be disabled while keeping the fur rendering as close as possible to the previous version. The upgrade process in XFur Studio™ 4 is always non-destructive.



A model using XFur Studio™ Core, successfully upgraded to XFur Studio™ 4 – Core Edition

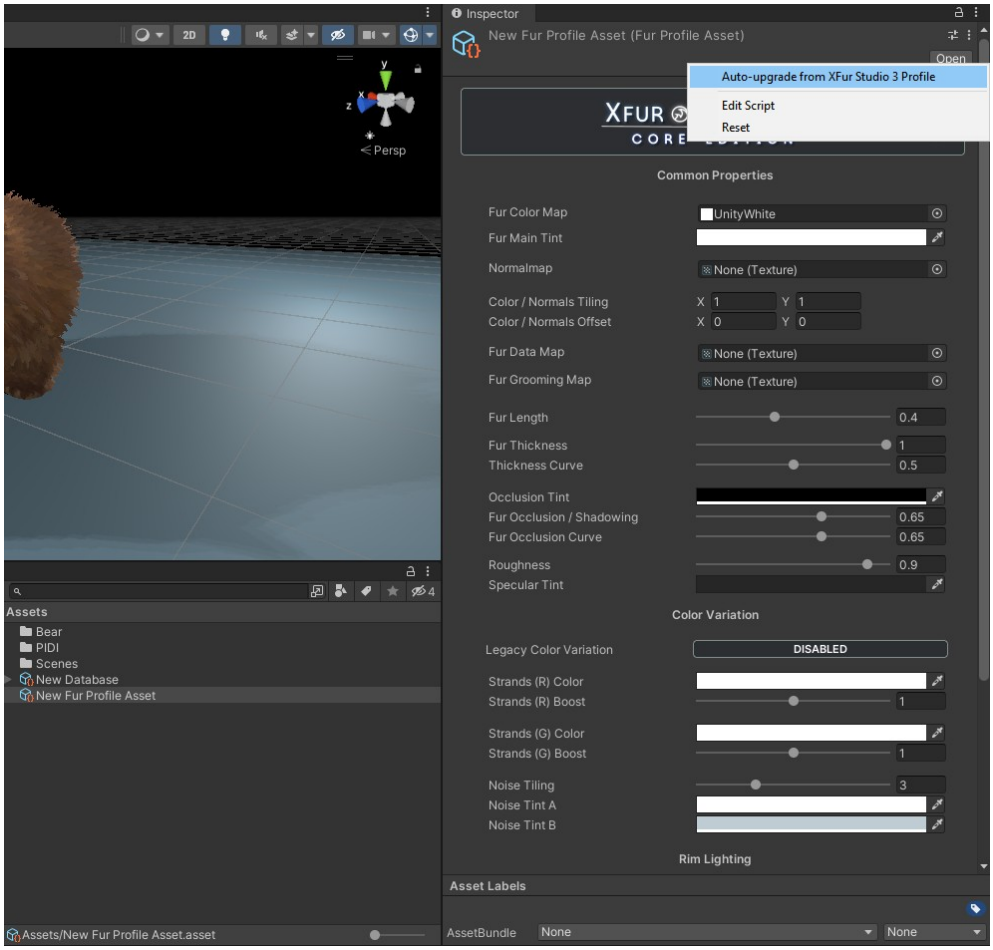
Important Notice: A few changes have been made to how color variation is handled, which means some tweaking may be necessary to achieve a 1:1 look. Our new color variation options offer much higher control over the final appearance of the fur however, so you may not want to revert back to XFur Studio™ 3 standard color variation setup.

To upgrade Fur Profiles, first create a brand new XFur Studio™ 4 – Fur Profile Asset anywhere in your project.



Creating a new Fur Profile Asset for XFur Studio™ 4

Once created, select it and right click on its inspector to bring up the context menu. Then click on the “Auto-Upgrade XFur Studio 3 Profile” option.



Upgrading an XFur Studio™ 3 Fur Profile Asset by copying its information to a new XFur Studio™ 4 Fur Profile Asset

This will display an “Open File” dialog from where you can select the XFur Studio™ 3 Fur Profile that you want to upgrade. Once selected, the XFur Studio™ 4 Fur Profile Asset will copy all the relevant content from the XFur Studio™ 3 one.

Important Notice: While upgrading, the actual texture from Fur Strand Assets from version 3 will be referenced by the version 4 Instances and profiles, ensuring that the fur appearance is preserved.

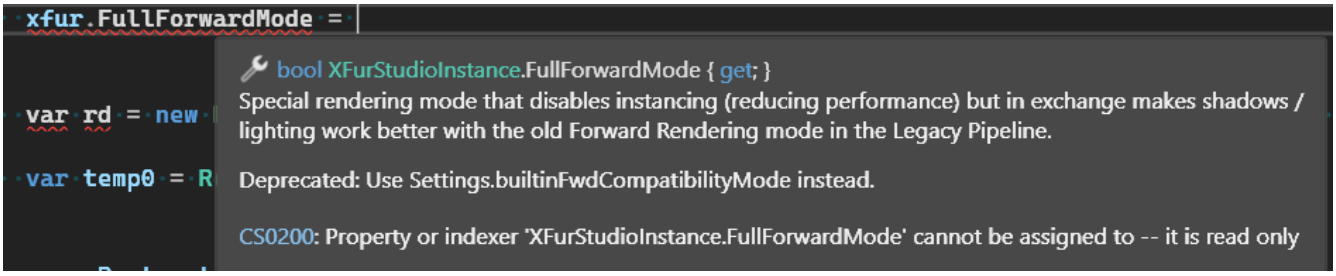
Fur Strands Assets can be upgraded in exactly the same way. You create a Fur Strands Asset for XFur Studio™ 4 first, then select it and right click on its inspector. You select the Auto-Upgrade option and then the old Fur Strands Asset that you wish to upgrade.

Upgrades to the XFur Studio™ API, internal functions and variable names

There have been extensive changes made to XFur Studio™ internal API for XFur Studio™ 4. While these changes provide a much easier and intuitive access to the asset's features and functions, they may break code designed for version 3.

To ensure an easy upgrade process, as long as version 3 is installed in your project concurrently with version 4 and the appropriate upgrade extensions have been unpacked, all of the old API commands and properties for the different classes should work with minimal changes, while internally referencing the new API and entry points.

Through the inline comments and documentation (available in your preferred IDE) you will see the updated APIs and property names, so that you can update your code at your own pace.



Example of an old API call still working, while providing details on the new, recommended API call.

A full breakdown of the API and the public properties and methods for each class is available in this documentation.

Setting up your project for XFur Studio™

There are a few requirements that all models intended for use with XFur Studio™ 4 have to follow. While models that do not follow these requirements may still work with our system it could cause different errors down the line, either with simple features such as Fur Grooming or advanced ones, like physics.

To ensure that your models will work in optimal conditions with XFur Studio™ 4 make sure that they:

- Are marked as Read/Write enabled on their import settings.
- Have a uniform (1,1,1) scale when imported into Unity. This applies to both the Rig and the actual Mesh that it deforms.
- Their local rotation & position must be 0,0,0 or as close to this value as possible.
- The UV layout for your model should avoid, as much as possible, any overlapping islands. If your character has mirrored UVs (both sides of your mesh are overlapping to save texture space) this will break the grooming features, as the fur direction will not be properly stored. Physics / VFX and other effects that depend on precise mesh position / normals may also work incorrectly.
- A secondary UV layout can be used to control how the fur strands are distributed over the model's surface, but it is not necessary.

Many models provided in the Asset Store will, by default, fulfill all these requirements. If you are having problems, please check our troubleshooting guide at the end of this documentation or contact us with the details.

While it is not required, it is heavily recommended that you install the Burst package from the official Unity Repository through the Package Manager into your project.

When Burst is installed, XFur Studio™ can make use of most of its advanced features while providing the best performance possible. Motion Vectors Rendering, for example, requires Burst to be installed in the project.

XFur Studio™ - Main Components

XFur Studio™ 4 consists of several components that work together to render efficient, high quality fur. The main two components that you will use for your creatures are Fur Strands Assets, which describe the appearance of the fur strands through “dot” based textures, and the XFur Studio Instance which controls and renders the actual fur for a Mesh or Skinned Mesh Renderer.

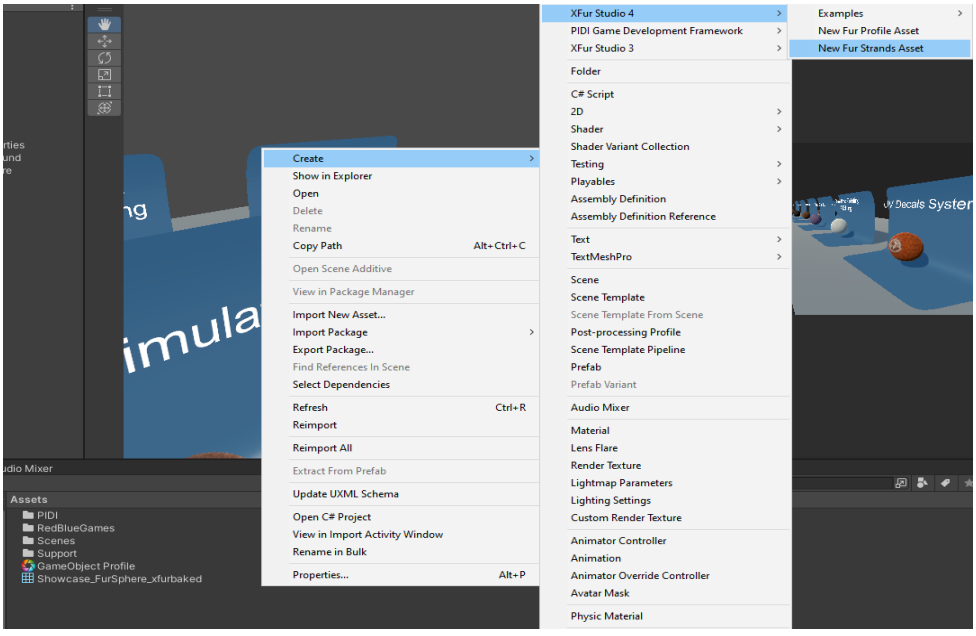
Fur Strands Asset

Fur Strands Assets are Scriptable Objects that contain a reference to a Strands Texture Map. These textures define the appearance, size and distribution of the “***strands***” that are displayed by the XFur Studio Instance.

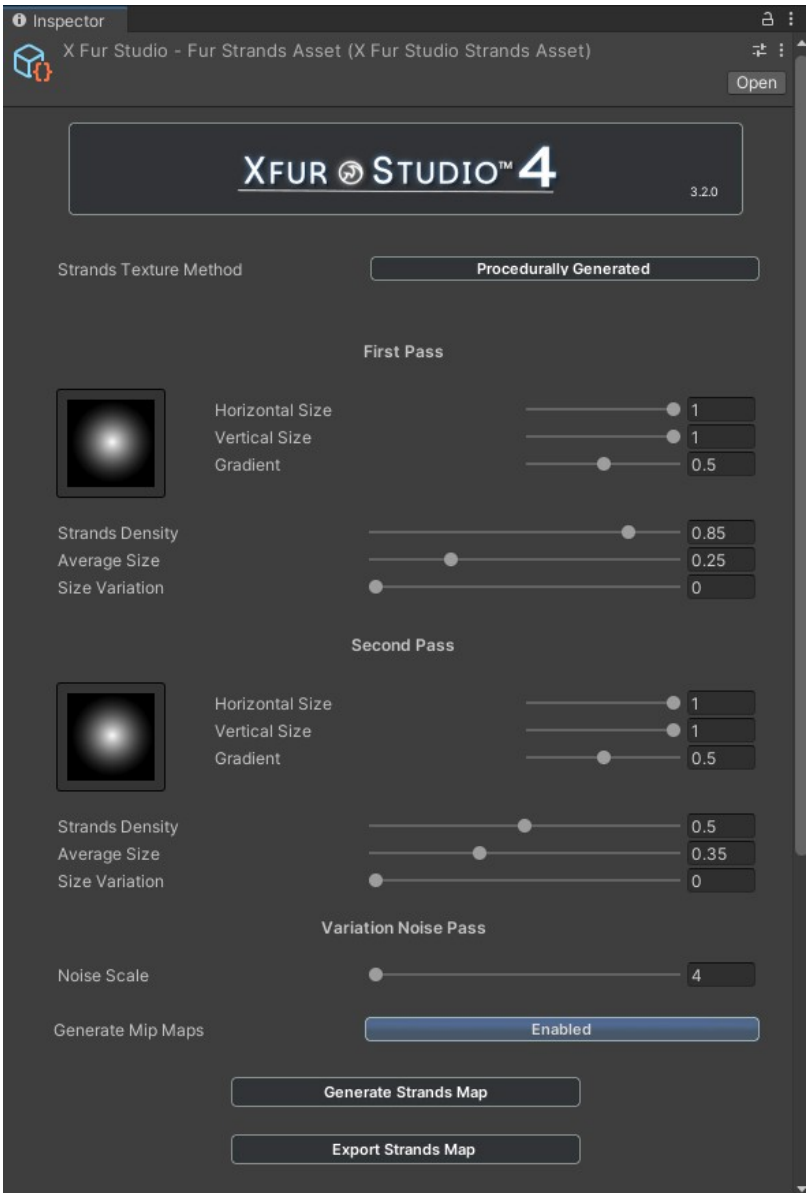
Important Notice: XFur Studio™ does not render fur through actual strands of simulated point-based or polygon-based geometry. Instead, it uses a shells-based approach. This method is faster on older hardware and more compatible with lower end devices as it requires neither compute nor geometry shaders.

These Strand Texture Maps can be generated or hand-made in the digital drawing software of your choice and then assigned to the Fur Strands Asset manually, or they can be procedurally generated by the Fur Strands Asset itself. The latter method is preferred, as it allows for faster iterations and ensures that the output is always in the format expected by the shader.

To create a new Fur Strands Asset, right click anywhere in your Project Panel and navigate to “Create/ XFur Studio 4/New Fur Strands Asset”



The newly generated Fur Strands Asset presents plenty of options:



- **Strands Texture Method:** Controls whether the Strands Map will be procedurally generated or a manually assigned external texture.
- **First Pass Settings:** Controls the appearance and distribution of the "main" set of strands.
- **Second Pass Settings:** Controls the appearance and distribution of the "second" set of strands.
- **Variation Noise Pass:** These settings control an additional blue channel Perlin Noise pass, to help with color variation control over the fur.
- **Generate Mip Maps:** Enables the mip-map generation in the resulting texture.
- **Generate Strands Map**: Generates the procedural Strands Map Texture and saves it as a Sub-Asset.
- **Export Strands Map:** Once generated, the resulting map can be exported as a PNG texture to be edited in an external software.

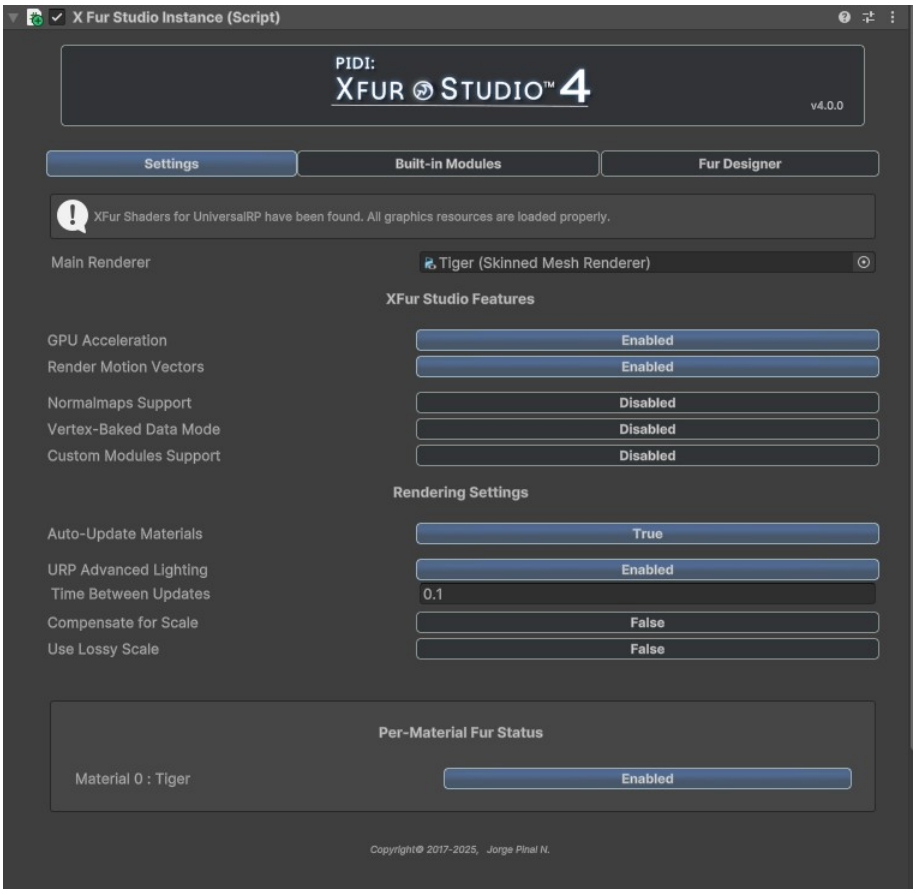
XFur Studio Instance

The XFur Studio Instance component is the main script that handles fur rendering for a given model. From it you can access all rendering settings, modules, features and the XFur Studio™ Designer window, to style the fur within the Unity Editor.

Important Notice: The XFur Studio Instance component is fully compatible with both Skinned and Static meshes. **It is not compatible with Alembic meshes**. It also requires a **GameObject.**

There are three main panels to the UI of the XFur Studio™ Instance Inspector: Settings, Built-in Modules and Fur Designer.

Settings Panel



- **Main Renderer:** This is the renderer that will display the fur. It can be either a Skinned Mesh Renderer or a Mesh Renderer. For models with multiple LODs you must select the LOD0 renderer.
- **GPU Acceleration**: Enables the use of Vertex Buffers to transfer the data from your mesh to the fur. This ensures that the skinning applied to your model is transferred through the GPU, rather than with the more costly (and CPU-based) Bake command.
- ***Render Motion Vectors:** For the Universal Rendering Pipeline (Unity 6+) and the High Definition Rendering Pipeline (Unity 2021+), XFur Studio™ now offers full support for Motion Vectors. This ensures that Temporal Anti-Aliasing, Motion Blur and other effects work flawlessly out of the box.
- **Normalmaps Support:** Enables the use of normal maps with the fur itself. This is in most cases not necessary as it can make the fur look strange (as it removes some of its softer, fluffier look).

- ***Vertex-Baked Data Mode:** Forces XFur Studio™ to read the fur styling and grooming data from the vertices of the mesh rather than from textures. This saves memory, but requires you to bake your styling choices through XFur Studio™ Designer.
- ***Custom Modules Support:** Enables the use of custom modules to expand the functionality of XFur Studio™ at any point of the rendering process. They work similarly to the built-in modules, such as the Physics or the VFX module.
- **Auto-Update Materials:** Automatically applies any changes done to the fur every given seconds (defined in the **Time Between Updates** field)
- **URP Advanced Lighting:** Available in the Universal Rendering Pipeline only. Switches between the default PBR based lighting used in the Lit material and a custom, more realistic model with support for anisotropic highlights.
- **Compensate for Scale:** Applies small adjustments to the fur length, thickness and other values to account for the scale of the model (useful for characters / creatures that are too small or too big).
- **Use Lossy Scale:** Whether to use Local or Lossy Scale values as a reference when compensating.
- **Per-Material Fur Status:** Whether a given material of the renderer will display fur or not. Useful to make sure that some materials (clothes, eyes, etc) do not render fur.

*** Not available in XFur Studio™ 4 – Core Edition.**

Built-in Modules Panel

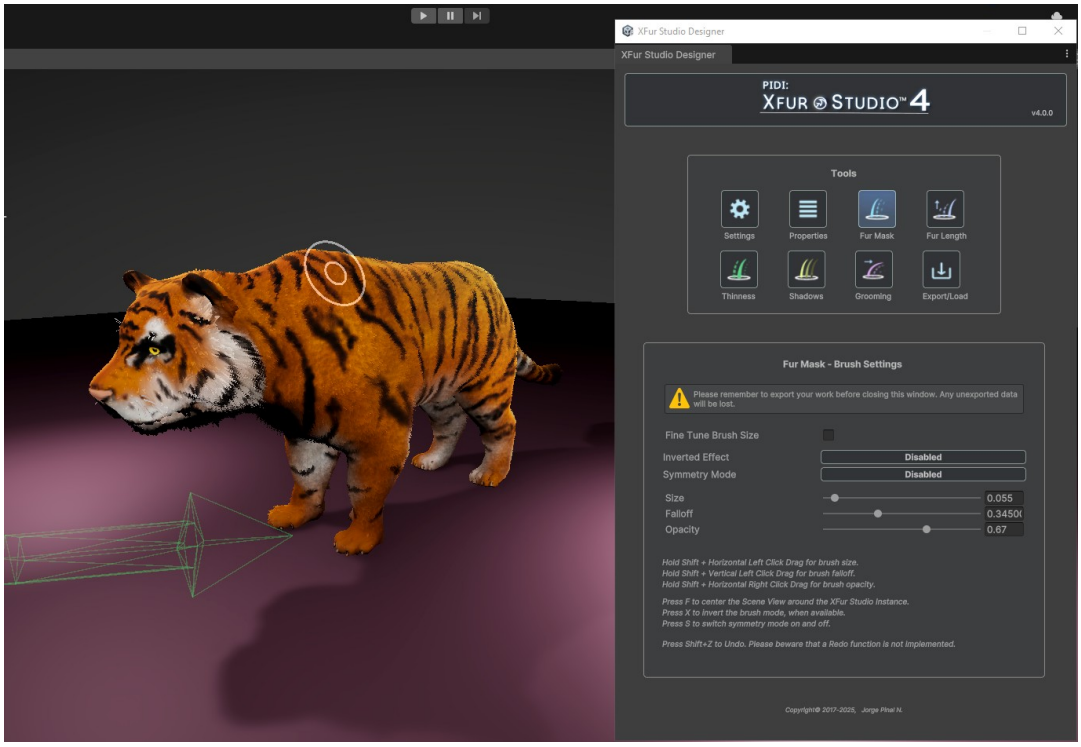
The actual functionality of each Module as well as how to use them will be discussed in detail in the Advanced Topics section of this documentation.

In the Built-in Modules tab you can enable or disable the different modules included with every XFur Studio™ Instance. All modules are compatible between themselves, and their status is displayed in the inspector itself.

Stable modules have been thoroughly tested, **Beta** modules are ready to use but may present some issues and require further testing, while **Experimental** modules are an introduction to potential new features that may or may not become a part of the full XFur Studio™ suite, and should not be used in a full production. All built-in modules are, on release, considered to be Stable.

Fur Designer Panel

All fur editing, styling and grooming is done through a separate in-editor window called XFur Studio™ Designer. To launch this window you select the Fur Designer tab in your XFur Studio Instance, pick which material to edit from the drop-down menu and then click on the Enter Edit Mode button.



XFur Studio Designer is a complex and fully featured set of tools, and as such it will be covered in detail in the "**XFur Studio Designer**" section of this documentation.

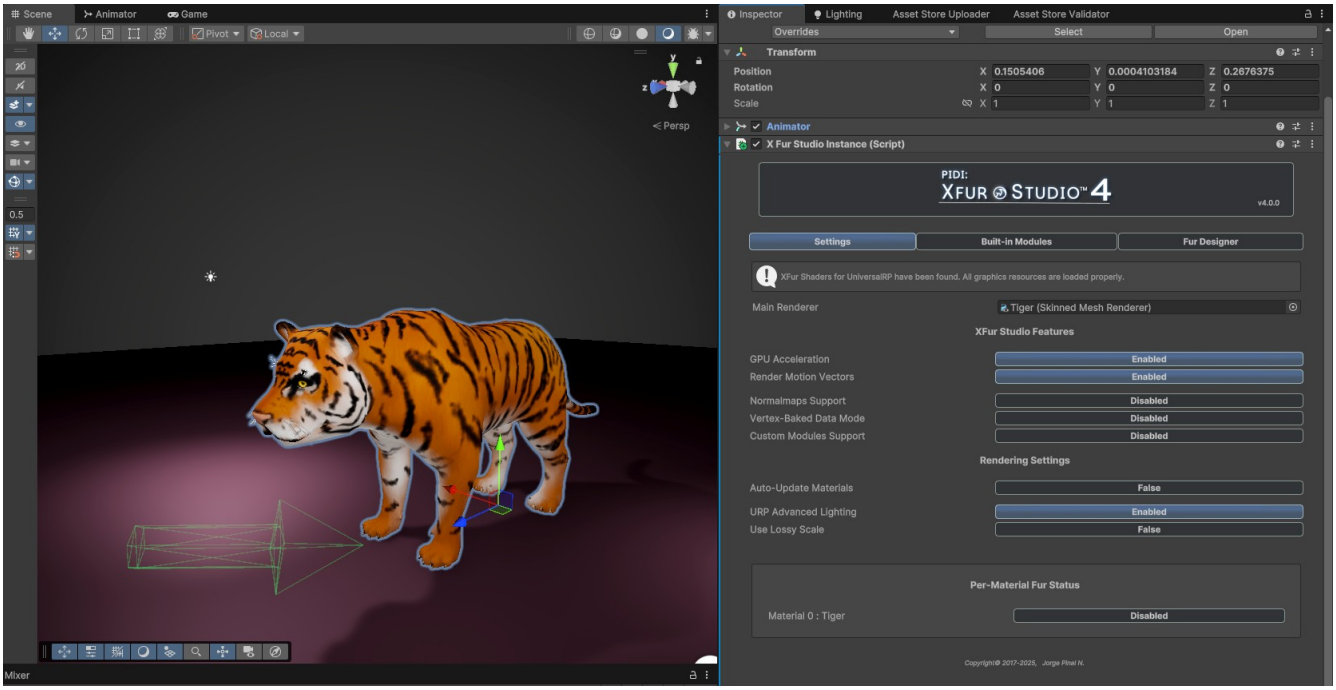
XFur Studio™ Designer

To better understand how XFur Studio™ Designer works and to get a hands-on experience using it, we have decided to approach this section of the documentation as a small tutorial.

If you are using XFur Studio™ Core, a small 3D model called “Eustace the Cat” is included with the asset.

If you are using XFur Studio™ 4 – URP, Personal or Team Editions you can use the Tiger demo model included with the asset. For this guide, we will use the Tiger model.

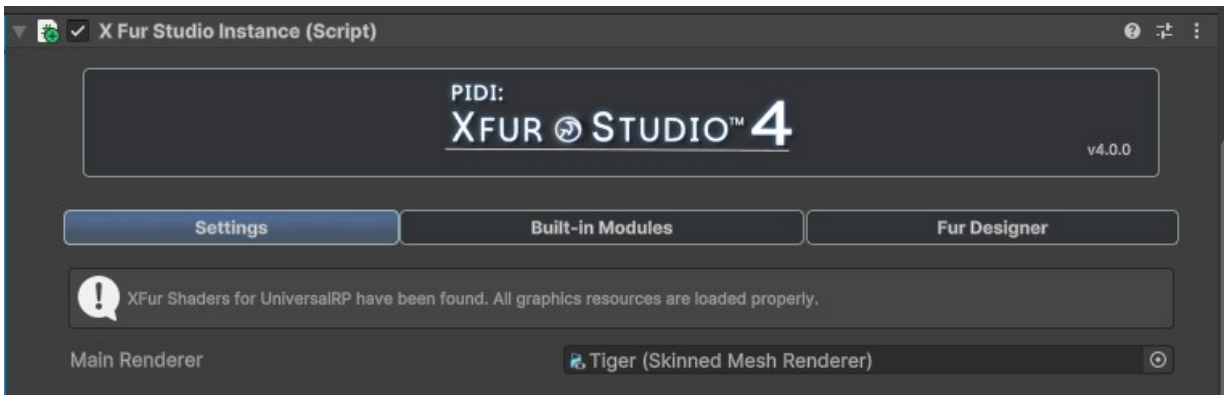
In an empty scene, drag and drop the Tiger model from the Demos/Tiger folder, then add the XFur Studio Instance component to it.



The Tiger model with the XFur Studio™ Instance component attached.

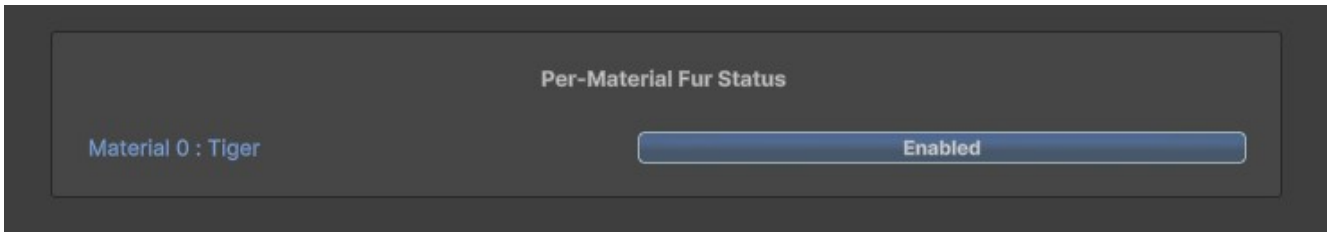
Once added, the component will automatically detect the first renderer it can find (in this case, the correct Tiger Skinned Mesh Renderer component).

A small box will indicate whether all the resources for the current rendering pipeline have been loaded successfully:

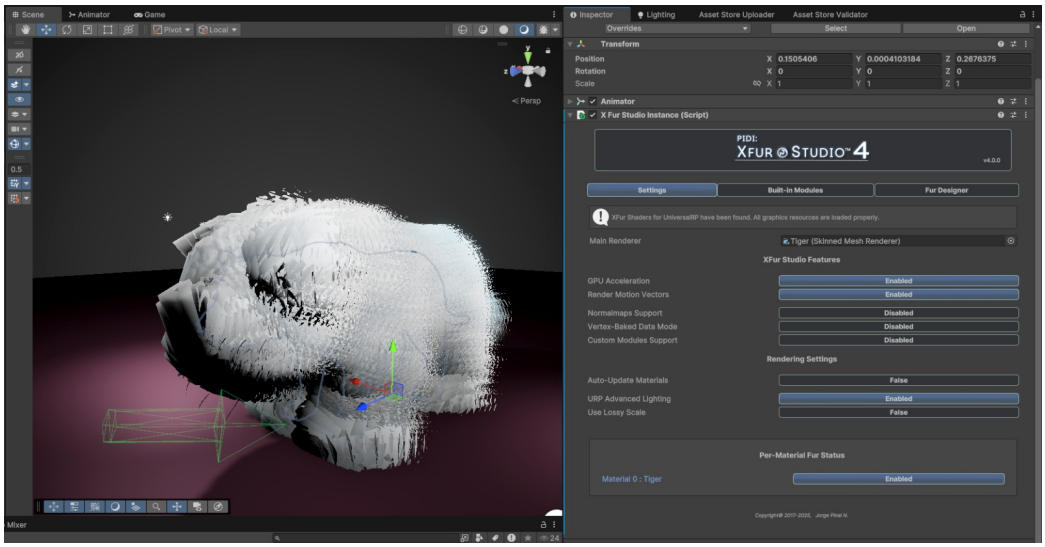


If they have been loaded successfully, proceed now to ensure that GPU Acceleration is enabled and, if available, the Render Motion Vectors setting is enabled as well.

Once this is done, enable fur for the main material in the tiger model.



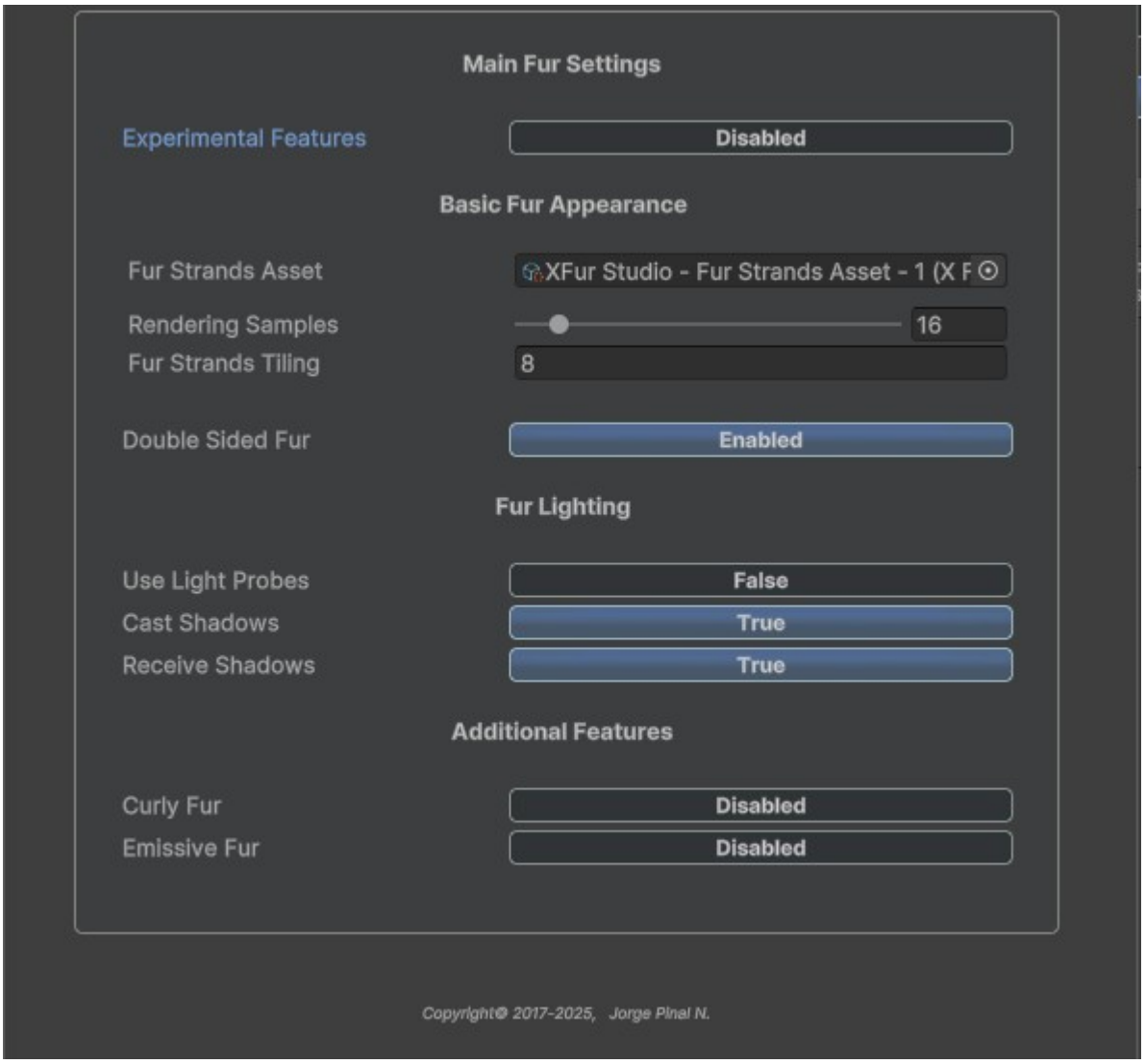
The fur of the tiger will not look correct at first. This is to be expected, as it has not been configured at all.



Let's open now the Fur Designer tab, select the Tiger material and Enter Edit Mode. The main XFur Studio™ Designer window is composed of two main panels, the Tools Panel and the actual inspector for each tool.

In the actual tools panel we have the Settings and Properties tabs, as well as brushes for the fur's mask, length, thinness, shadows and grooming. An extra tab to Export and Load the styling is also available. We will check each tool, one by one.

Settings Tool

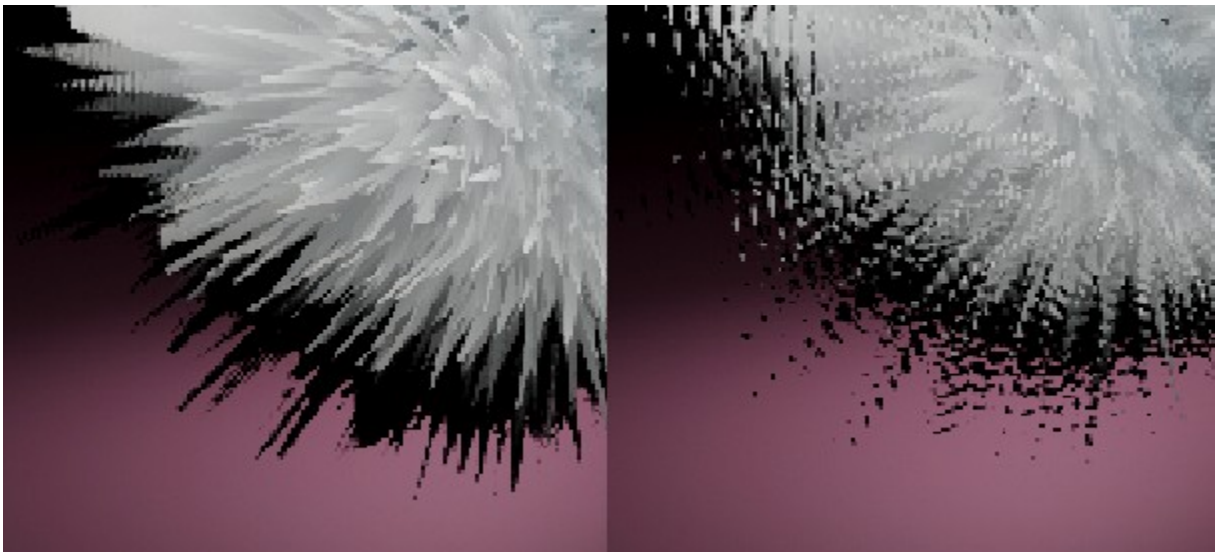


This is the Settings tool. With it we control the general settings for the rendering of the fur of our Tiger. We can also enable / disable different rendering and styling features.

- **Experimental Features:** With this setting we can enable the use of experimental rendering features. At launch, XFur Studio™ 4 has no such features, but they may become available in later releases.
- **Fur Strands Asset:** The asset that defines the appearance of the strands. For this tutorial, we will select the Fur Strands Asset – 2.

- **Rendering Samples:** Controls the amount of samples or shells of fur that will be rendered. More shells produce higher quality fur strands at the cost of performance. Each shell renders the model one more time, increasing polygon counts. Always keep the poly counts and rendering samples of fur covered meshes as low as possible.
- **Fur Strands Tiling:** The tiling applied to the Fur Strands Map. The higher the tiling, the thinner each strand will appear and the fuller the fur will look.
- **Double Sided Fur:** Renders the fur as a double-sided mesh.
- **Use Light Probes:** Whether the fur will support the use of light probes in the scene. This is limited by the compatibility between Unity's Light Probes and GPU instancing.
- **Cast Shadows:** Whether the fur will cast shadows or not.
- **Receive Shadows:** Whether the fur will receive shadows or not.
- ***Curly Fur:** Enables the settings to generate curly fur by twisting the strands as they get longer.
- ***Emissive Fur:** Enables the settings to add emission effects to the fur itself.

***Not available in XFur Studio™ 4 - Core Edition**



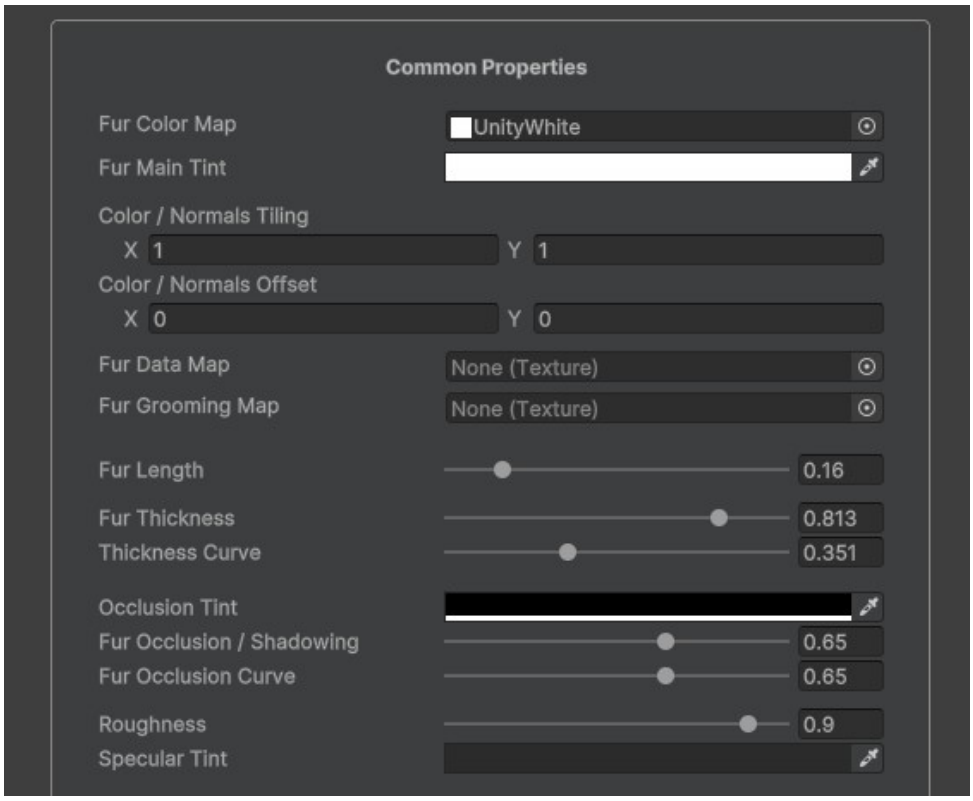
A comparison between high Rendering Samples counts and Low Rendering Samples counts

For our tiger, we will set the Fur Strands Tiling to 16 and the Rendering Samples to 16. We will leave all the other settings untouched.

Let's switch now to the Properties Tool.

Properties Tool

The properties tool is split into several categories.

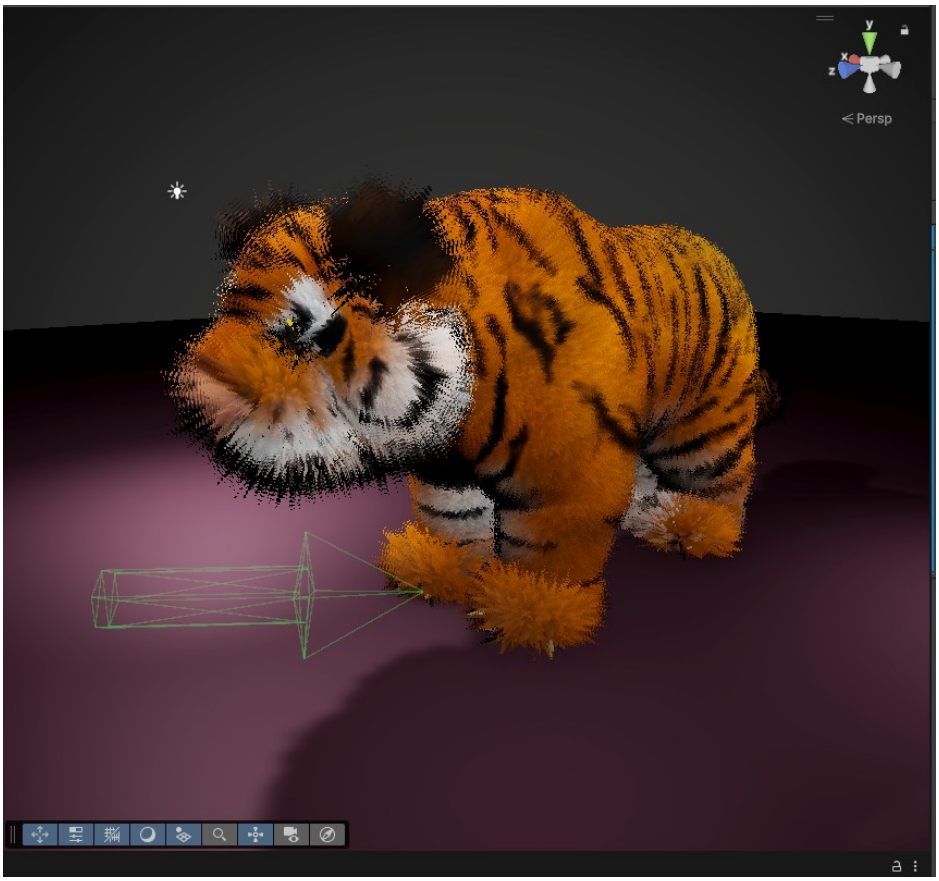


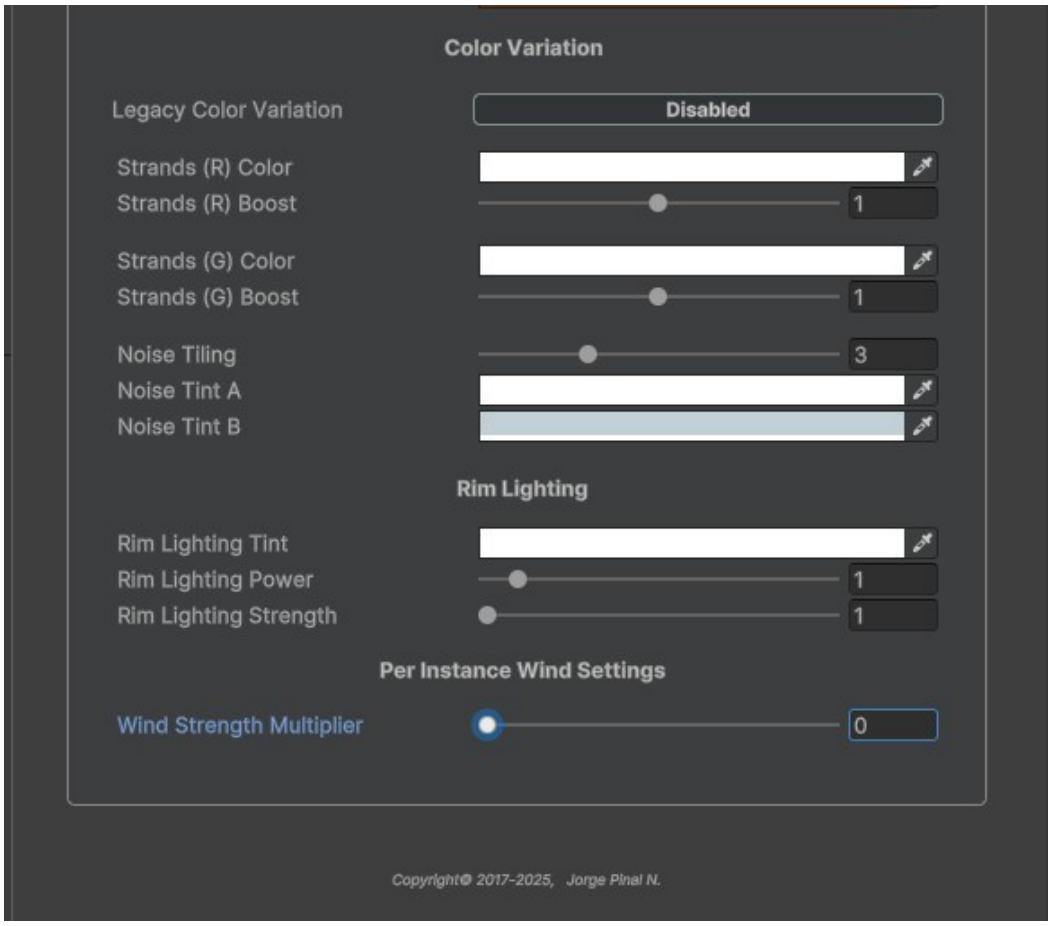
Common properties control the main tint, styling and grooming of the fur.

- **Fur Color Map:** It is the equivalent of the Albedo map in the Standard / Lit shader. We will assign our Tiger's albedo map here.
- **Fur Main Tint:** Applies a tint to the fur's color. In this tutorial, we will leave it white.
- **Color / Normals Tiling and Offset:** The tiling and offset to be applied to the UVs of our model.
- **Fur Data Map:** The texture map containing the per-vertex styling of our model (fur length's, thickness, occlusion, etc). While a map is provided for the Tiger model, we will create our own in this tutorial.
- **Fur Grooming Map:** The texture containing the per-vertex grooming for the fur of our model. While a map is provided for the Tiger model, we will create our own in this tutorial.

- **Fur Length:** Controls the overall length of the fur. It should be as long as the longest section of the fur will be. For this tutorial, we will set it at 0.16.
- **Fur Thickness:** Controls the overall thickness of the fur strands. We will leave it at 1.
- **Fur Thickness Curve:** Controls how the thickness of the fur changes from the root to the tip of each strand. We will set it at 0.4.
- **Occlusion Tint:** The tint of the shadowing / occlusion applied to the fur. We will leave it as is.
- **Fur Occlusion / Shadowing:** The intensity of the shadowing applied to the fur. It is modulated by the blue channel of the Strands map. We will set it at 0.65.
- **Fur Occlusion Curve:** The intensity of the shadow as it goes from the root to the tip of a fur strand. We will set it at 0.5.
- **Roughness:** How shiny the fur will be. We will set it at 0.9.
- **Specular Tint:** Controls the color and therefore the intensity of the specularity / shininess of the fur. We will set it to a nice dark orange RGB(100, 50, 15).

At this point, our tiger should be looking a lot better already:





Color Variation controls the way that the fur's tint and color will change from one part of the creature to the next. It has two modes, Legacy and Standard.

The Standard mode, enabled by default, adds certain randomization and noise passes to make the fur look more natural without any additional textures.

The Legacy mode uses an additional texture to define up to four different tints to be mapped to each of the texture's color channels. For this tutorial, we will leave the Legacy Color Variation mode disabled.

Now let's go over the next settings one by one.

- **Strands (R) Color:** The color to be applied to the red strands (red dots) in the Strands Map. We will leave it white.
- **Strands (R) Boost:** Multiplies the color of the red strands by this number, allowing you to make them lighter or darker than the rest of the fur. Set it to 1.1.

- **Strands (G) Color:** The color to be applied to the green strands (green dots) in the Strands Map. We will leave it white.
- **Strands (G) Boost:** Multiplies the color of the red strands by this number, allowing you to make them lighter or darker than the rest of the fur. Set it to 2.
- **Noise Tiling:** The tiling applied to the additional noise pass that modifies the tint of the fur. Set it to 2.75.
- **Noise Tint A:** The main color applied to the white parts of the noise. Leave it white.
- **Noise Tint B:** The color applied to the black parts of the noise. We will use a “dirty” yellow tone RGB(210,190,150).

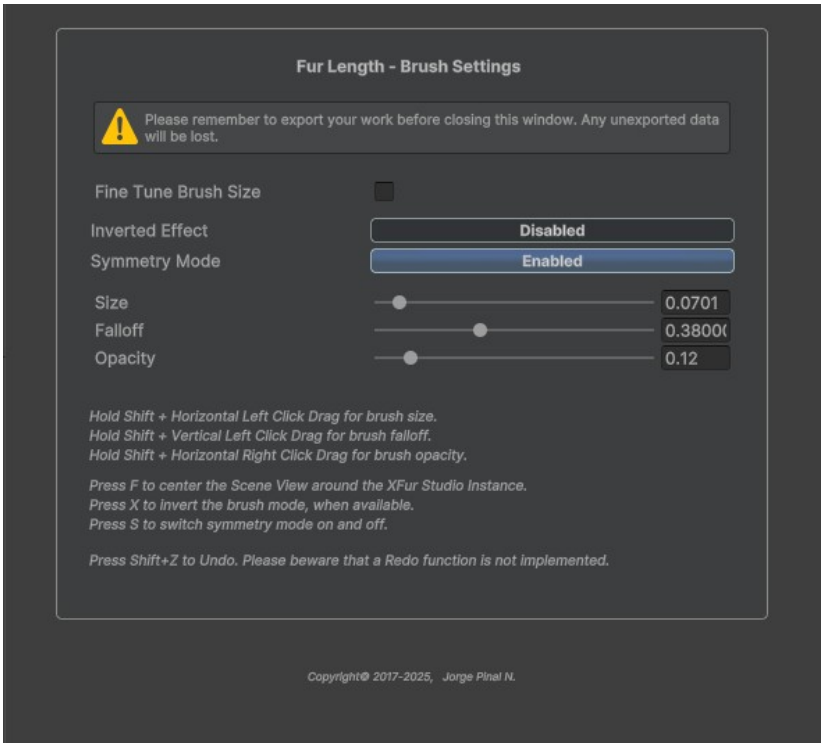
The **Rim Lighting** settings allow you to simulate a bit of transmittance on the fur while making it look fluffier.

- **Rim Lighting Tint:** The color used for the Rim Lighting Effect. We will use a light peach color RGB(255, 200, 150).
- **Rim Lighting Power:** Controls the thickness / thinness of the rim effect around the model. We want a thin rim, so we will set the value to 3.8.
- **Rim Lighting Strength:** The strength of the rim lighting itself. We will set it to 1.4.

Finally, the **Per Instance Wind Settings** allow you to modify the strength of the wind effect applied to each instance. This way you can make the wind effect milder or stronger as needed, which can be useful for smaller or bigger creatures. For now, let’s set the **Wind Strength Multiplier** to 0. With that, our tiger will be looking a lot better already.

Styling the Fur

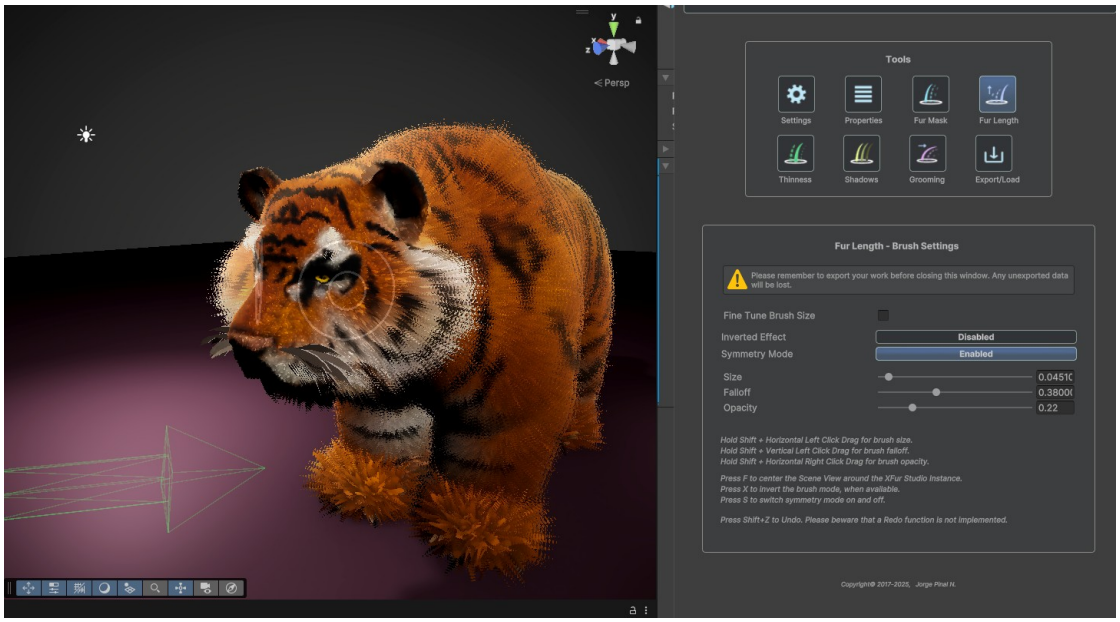
The next tool we will use is the **Fur Length** tool. This tool is an actual brush, and at this point we will learn how to style the fur in XFur Studio™ through the in-editor painting tools.



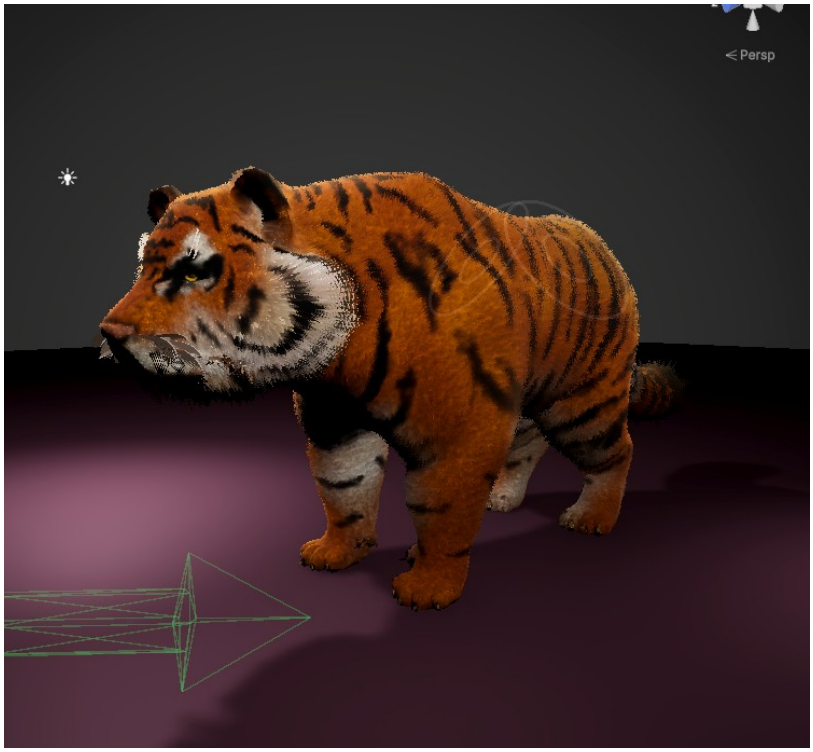
Each brush in XFur Studio™ Designer has several properties.

- **Fine Tune Brush Size:** Allows you to control the minimum and maximum size that the brush can have.
- **Inverted Effect:** Inverts the current brush. For masking, for example, it removes fur by default and adds it back when **Inverted Effect** is enabled. For fur length, it makes the fur shorter by default and grows it back to its default length when inverted.
- **Symmetry Mode:** Mirrors the brush on the local X axis so that both sides of the model are painted at once.
- **Size, Falloff and Opacity:** Control the size and strength of the effect applied through the brush.

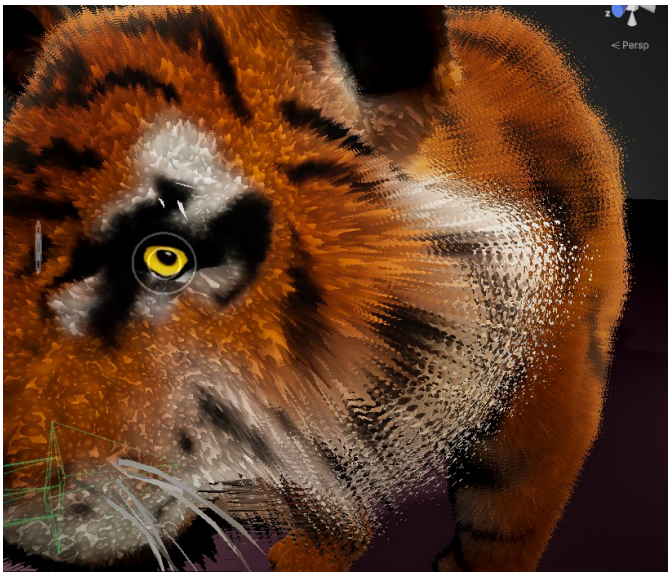
Using the brush for Fur Length with Symmetry Mode enabled and a very low opacity and falloff values, gently and slowly shorten the fur around the face of the tiger until its eyes and nose are fully visible.



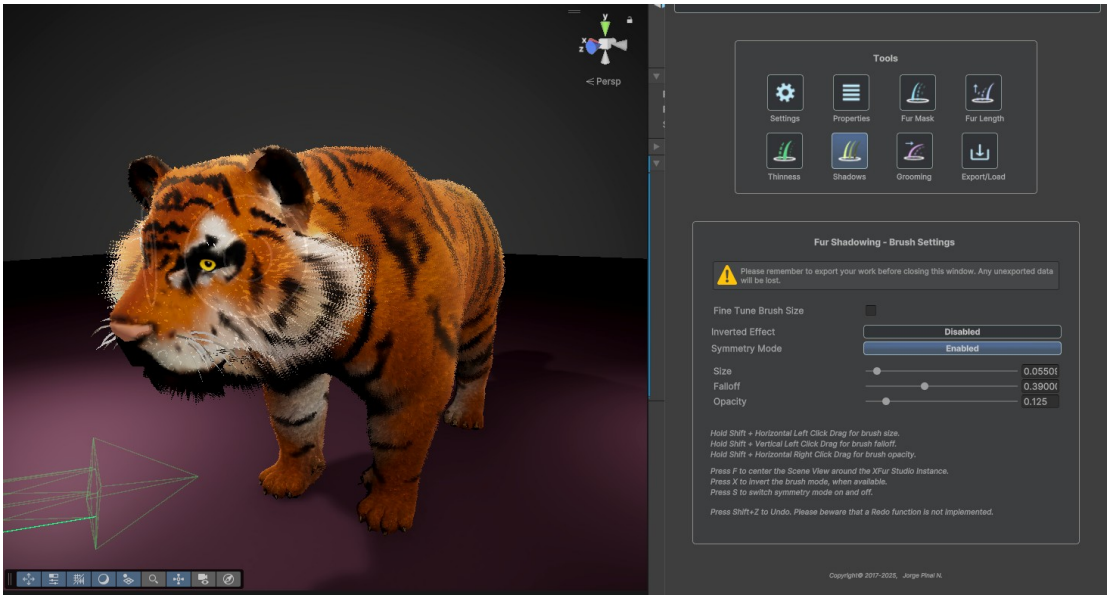
Keep working your way around the tiger until its fur is very short, almost invisible around the paws, and overall short across the body. Remember that tigers have long fur around their cheeks, so leave it long around that area. For reference, check the picture below.



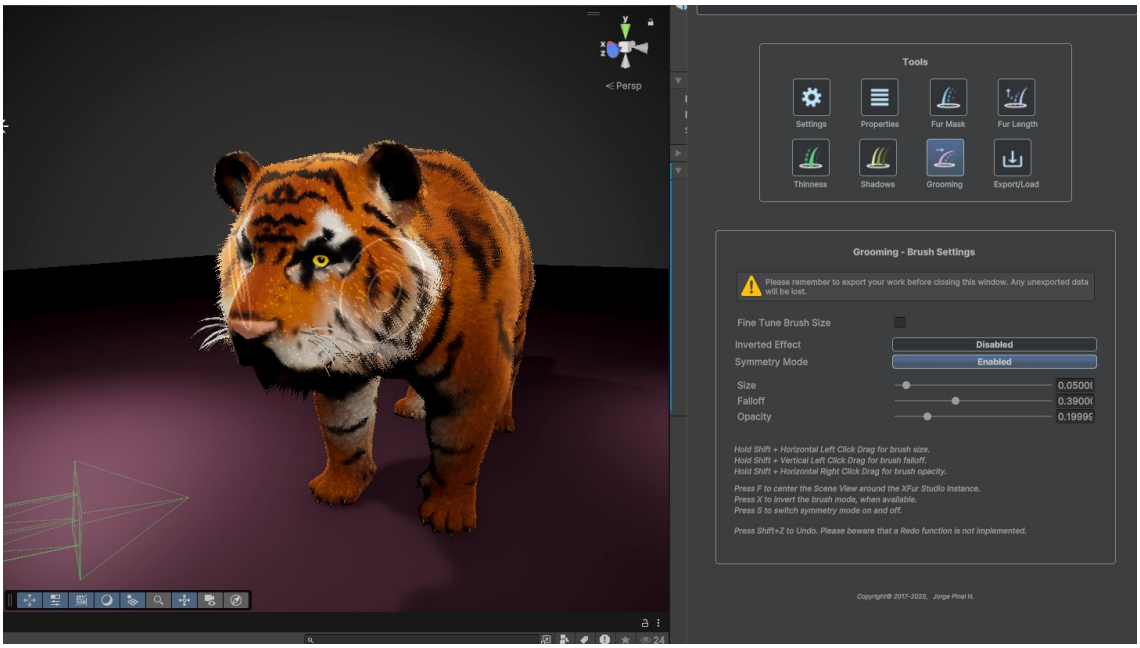
Using the Fur Mask tool in standard mode (not Inverted), remove the fur from the eyes and nose area, and try to remove it from the whiskers as well. If you struggle to select the whiskers, do not worry. We can clean the result by hand after we finish. If you remove fur from a wrong area by mistake, invert the brush and paint over it. This will make the fur grow back.



Next switch to the Shadows tool. Using it in Inverted mode, remove some of the shadowing from the face of the tiger. This will make the areas with short fur look better, as they will match the look of the mesh underneath more closely.

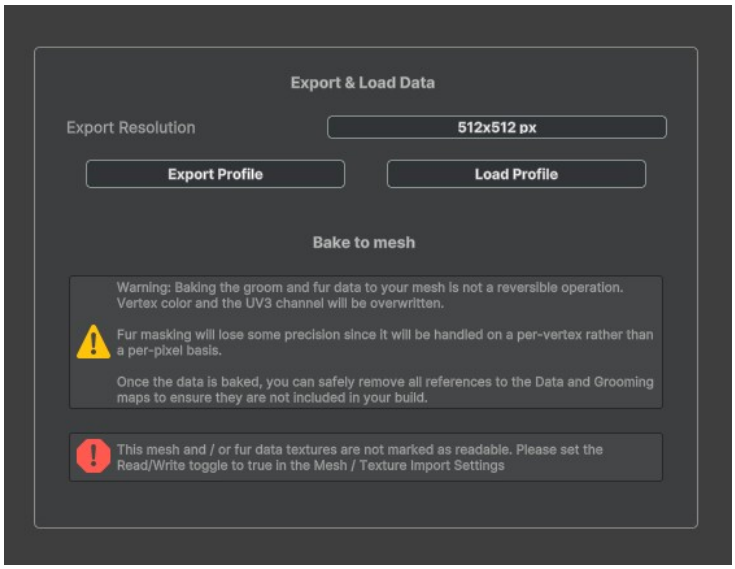


Finally let's use the Grooming tool. The Grooming brush allows you to comb the fur of your model and set its direction by stroking it directly. Stroke the fur around the face of the tiger downwards, and the rest of its fur following the natural flow of the body of the animal. The fur on the legs, downwards. The fur on its back in a direction from its head towards its tail, etc.



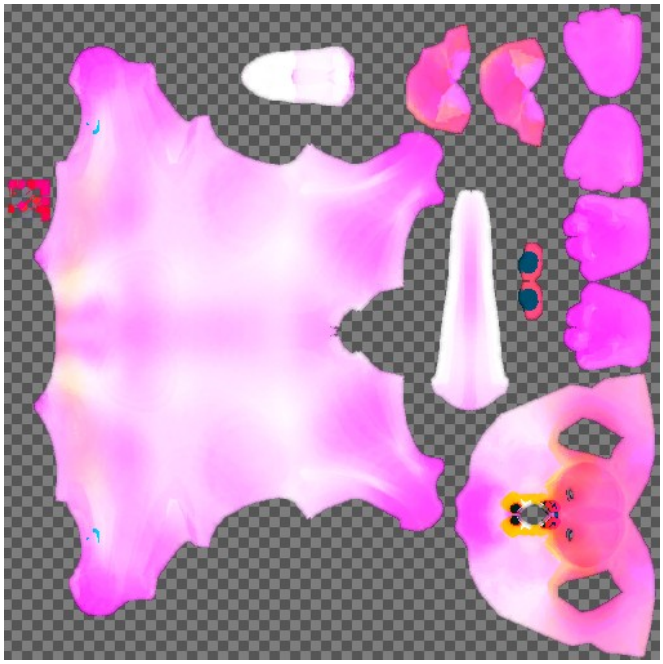
Once you are done grooming the fur of the tiger, feel free to go back and do any further adjustments with the brushes or with the properties themselves. Play around with the different brushes and settings until you have something you are happy with.

To finish this tutorial, let's switch to the Export / Load tool and export our work.



Here we have a few options. You can select the resolution of the exported maps, which is helpful when editing them in an external software. For now, let's export the profile with the default resolution.

XFur Studio™ Designer will export three files, a Fur Data Map, a Fur Grooming Map and a Fur Profile Asset. Open the Fur Data Map on the image manipulation software of your choice, and it will look something like this:



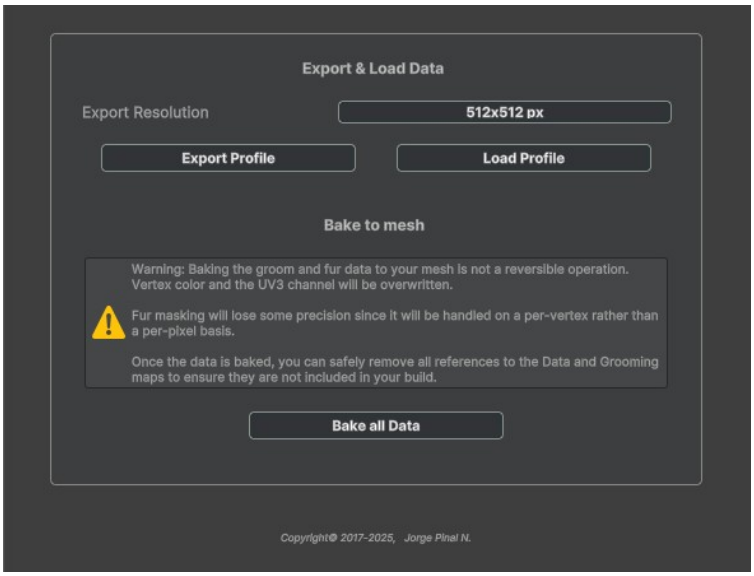
The small red squares on the left side are the whiskers. The white portion in the top center is the inside of the mouth. By painting them black, you will remove all fur from them.

In Fur Data Maps, red always controls the fur's mask. Green, its thickness and blue its shadowing. They can be easily manipulated in external software.

Fur Grooming maps on the other hand represent directions specially encoded in tangent space. As such, they can only be authored with XFur Studio™ Designer.

Baking Fur Data

If you mark both the Fur Data and the Fur Grooming maps as Read/ Write enabled in their import settings a new option will appear in XFur Studio™ Designer under the Export / Load tool:



Baking all Data allows you to transfer the fur styling and grooming from the textures to the mesh's UV3 and color channels. This operation will create a brand new mesh (all XFur Studio™ operations are non-destructive towards your original art) and automatically assign it to the Mesh / Skinned Mesh Renderer.

To switch to the Vertex-Baked Data mode, you must enable it in the main settings of the XFur Studio Instance Component.

Important Notice: Once Baked, the fur styling can no longer be modified through modules nor XFur Studio™ API commands unless a Fur Data Map and a Fur Grooming Map are provided again. The Physics, VFX, LOD, Decals and Simple Bending modules will work as intended.

You can also watch a video timelapse showing this tutorial on our Youtube channel.

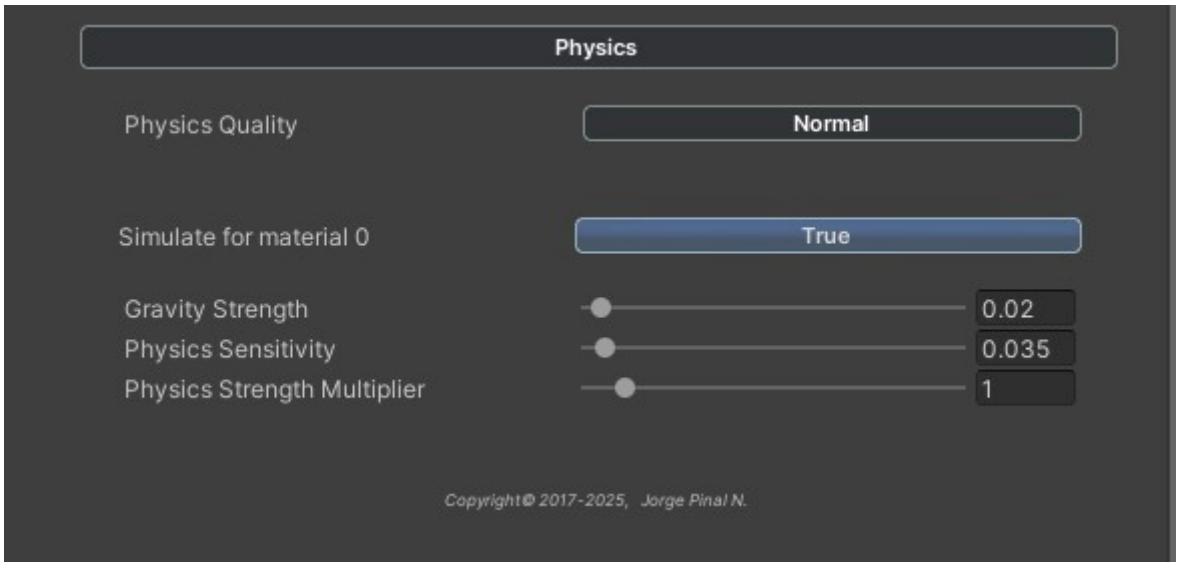
Built-in Modules

Important Notice: Modules are not supported in XFur Studio™ 4 – Core Edition.

Physics Module

The physics module allows you to simulate basic physics on the fur of your characters. It is entirely run on the GPU so its performance impact is extremely low, and its memory footprint can be adjusted by changing the quality of the Physics Simulation. Low and Very Low qualities will be enough for most models and in its lowest setting the Physics module uses less than 10Kb of memory for each material using the physics simulation.

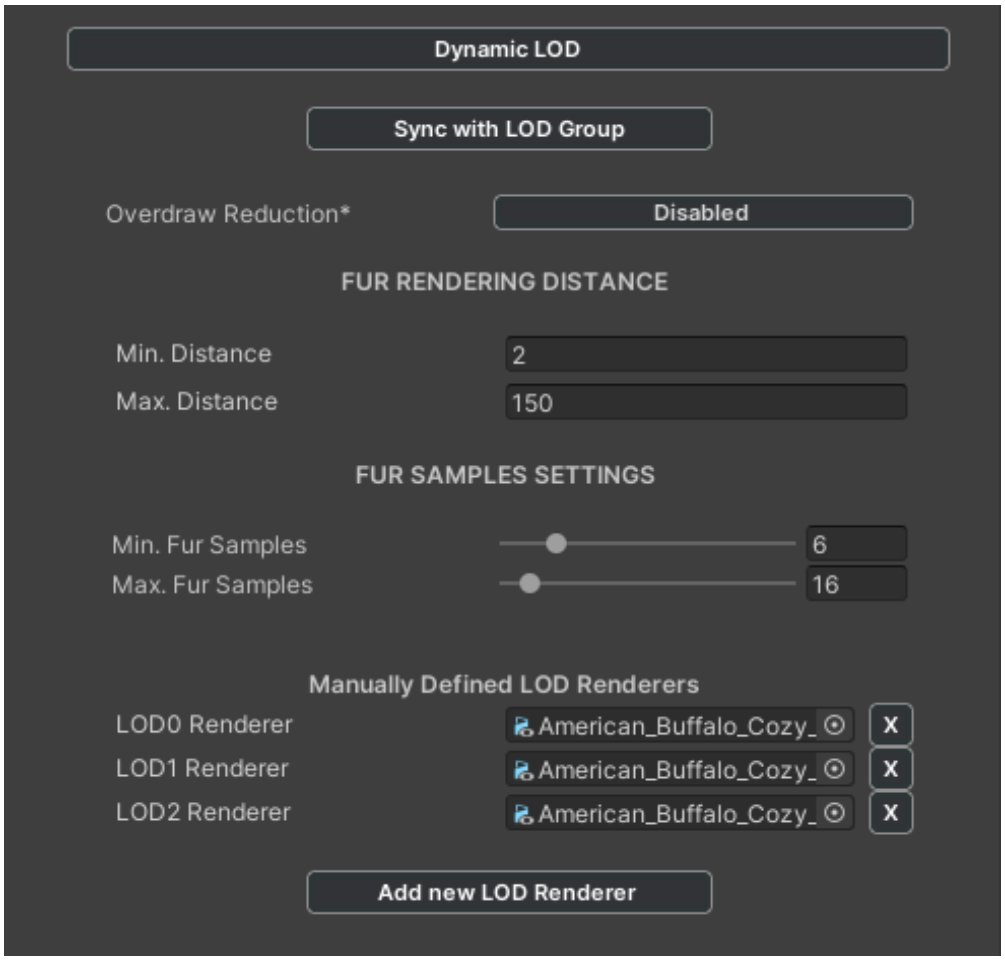
To use it, simply enable the module in the Settings tab and then enable the simulation for the corresponding fur materials in the Physics Module tab.



The different settings for Physics are shared across all fur materials. **Physics Sensitivity** controls how much the fur will respond to smaller changes in motion and animation, while the **Physics Strength Multiplier** adjusts the overall strength of the physics based motion.

LOD Module

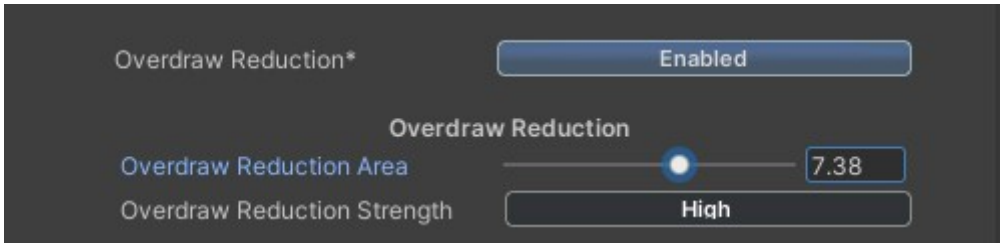
The Dynamic LOD module allows you to dynamically adjust the quality and features of XFur depending on the distance between the character and the camera. It is compatible with the standard Unity LOD Group component, adding fur to each active LOD level when needed and aiding you to keep a stable performance.



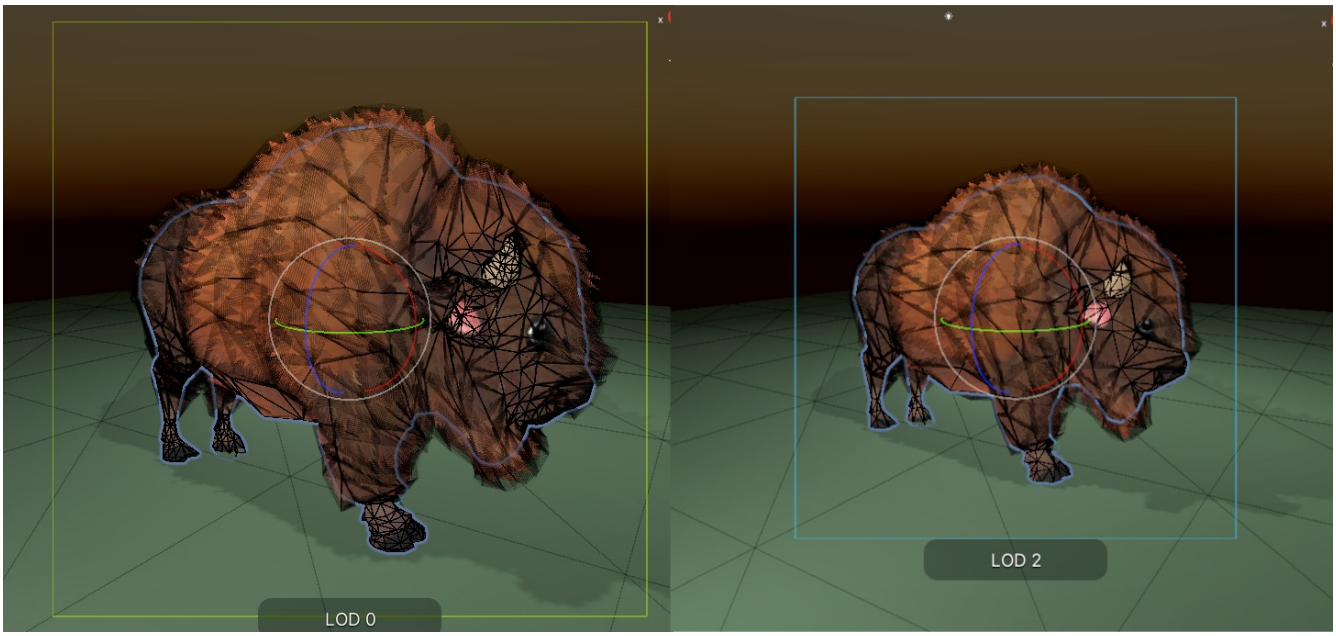
The component will ensure that the amount of fur samples goes from the **Min Fur Samples** amount towards the **Max Fur Samples** amount as the camera gets closer to the model, having **Min Distance** and **Max Distance** (measured from the model to the current camera) as the limits for fur rendering.

Pressing the **Sync with LOD Group** button will automatically find the LOD Group component attached next to the XFur Studio™ Instance component, read the first Renderer assigned to each LOD Level and automatically add fur to it when needed

Important Notice: The Sync with LOD Group feature works only on the first renderer attached to each LOD Level. If your character is made of multiple renderers or the mesh using fur is NOT the first renderer assigned to each LOD Level, you should not use the Synch with LOD Group button. Instead, you may add new LOD Renderers manually.



Another feature available with the Dynamic LOD module is Overdraw reduction. IT allows you to reduce the amount of passes of fur directly in front of the camera, where they are less noticeable, in order to improve performance slightly on older devices.

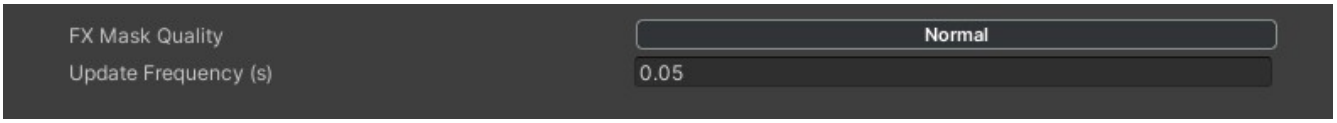


XFur Studio™ 4 working with the LOD Group component to sync the fur to each LOD Renderer

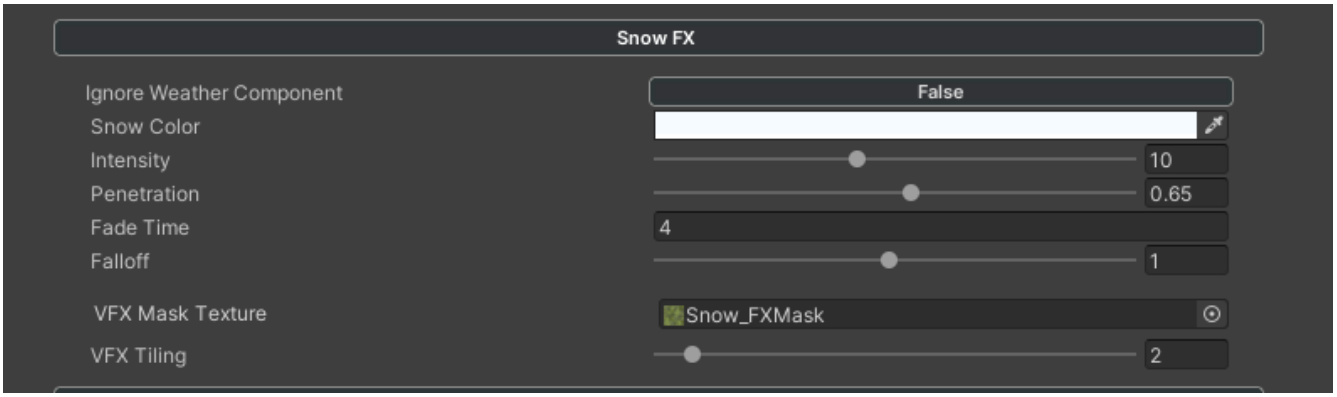
VFX Module

The VFX module lets you simulate snow coverage, blood effects and rain on the fur. Together with the **XFur Weather Manager** object, it allows you to simulate some weather effects over the fur such as wind, snow and rain. An additional VFX, blood, can also be simulated and painted dynamically over the fur either through the XFur Studio™ API or through the XFur Painter Objects.

A general quality for the VFX map can be specified. Lower qualities result in blurrier VFX but a much lower memory usage. The update frequency lets you control how often will the VFX map be regenerated.

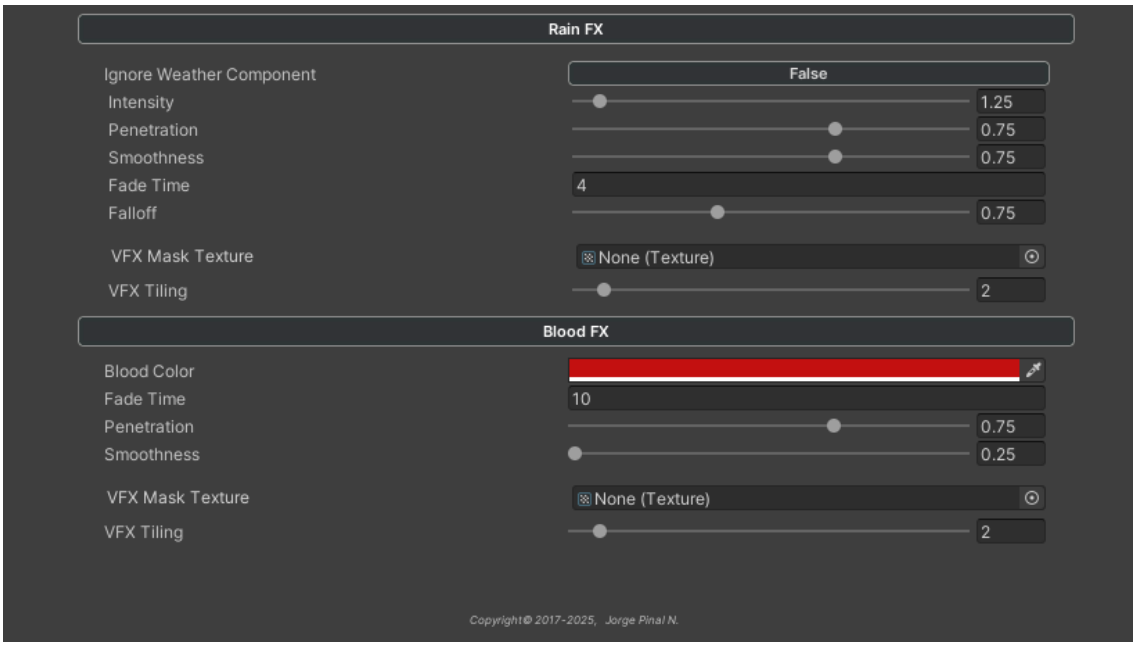


For each one of these VFX you can adjust different parameters, or decide to optionally ignore them entirely (so that certain fur enabled objects do not get affected by snow, for example).



For Snow, you can control its color, boost or diminish its intensity over the fur, control how much it penetrates through the strands, and how long it takes to melt away once it stops falling (once its intensity is set to 0 either in the VFX module or in the Weather Manager). You can also control the falloff of the snow coverage. Higher falloff values make the snow accumulate only on faces of your model whose normals are pointing opposite of the direction in which the snow is falling. Lower falloff values allow faces pointing further and further away from the snow to still get covered.

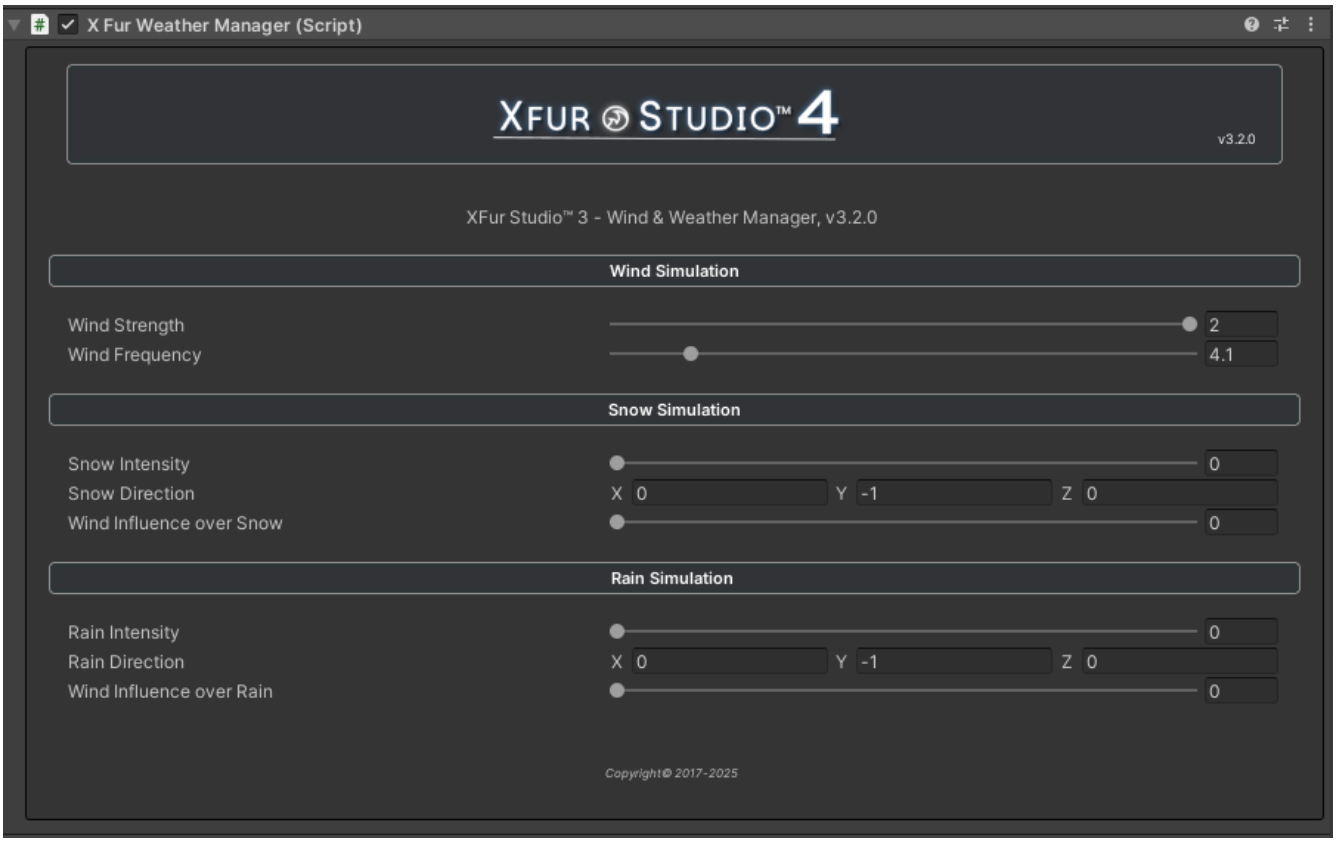
Finally, you can also specify a Mask texture to add variety to the snow coverage, and a custom tiling value for this mask.



Rain and blood offer similar amounts of control. Blood however does not have an intensity value, as it is not applied over time but rather in one-off bursts. Rain does not offer a color value, since rain only gets the fur wet (and darkens it slightly).

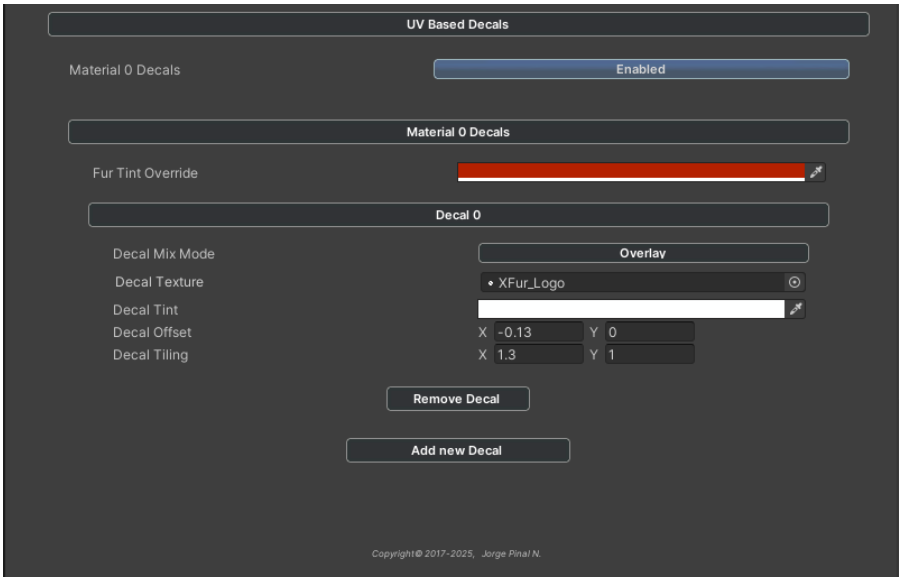
An example of a fur covered creature using the VFX Module with snow enabled.

The second component for VFX simulation is the XFur Weather Manager component. It should be attached to a singular object in your scene, since it acts as a global object that handles the weather simulation for all XFur Studio™ instances.

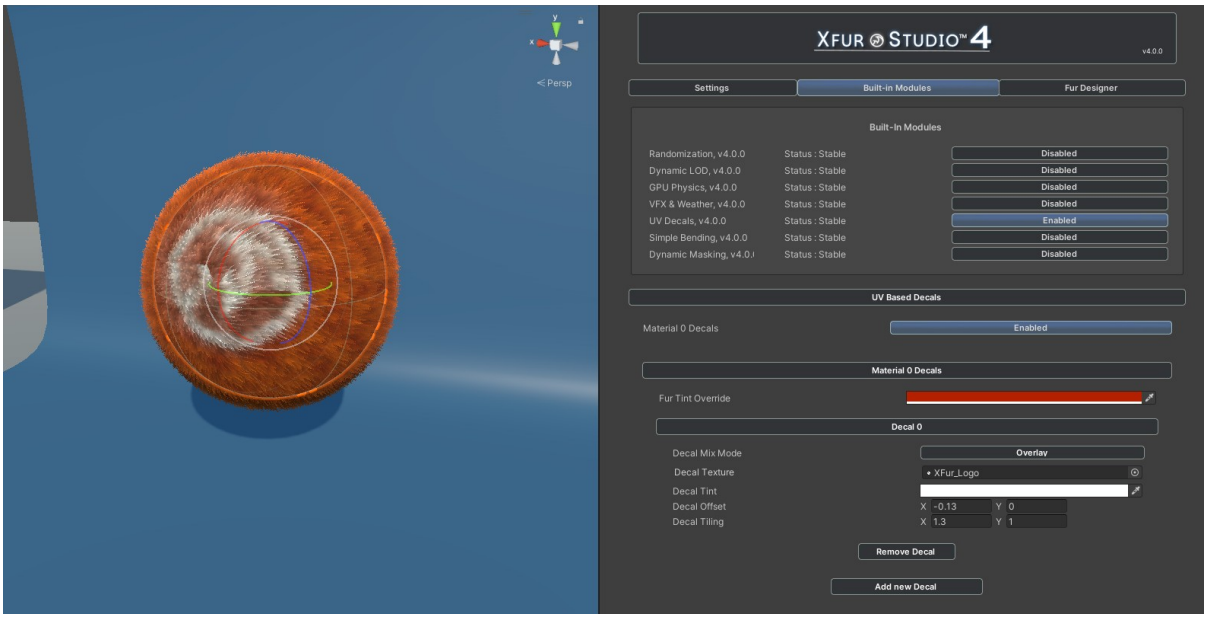


It provides with easy to use controls for Wind Strength and Frequency, as well as Snow / Rain Intensity and Direction, the latter of which can be influenced by the wind.

UV Based Decals Module



The Decals module allows you to project additional textures over your fur. Up to four decals per material can be applied and each one of them can be layered and mixed in overlay, additive or multiplied mode. These decals are projected using the main UVs of the mesh.



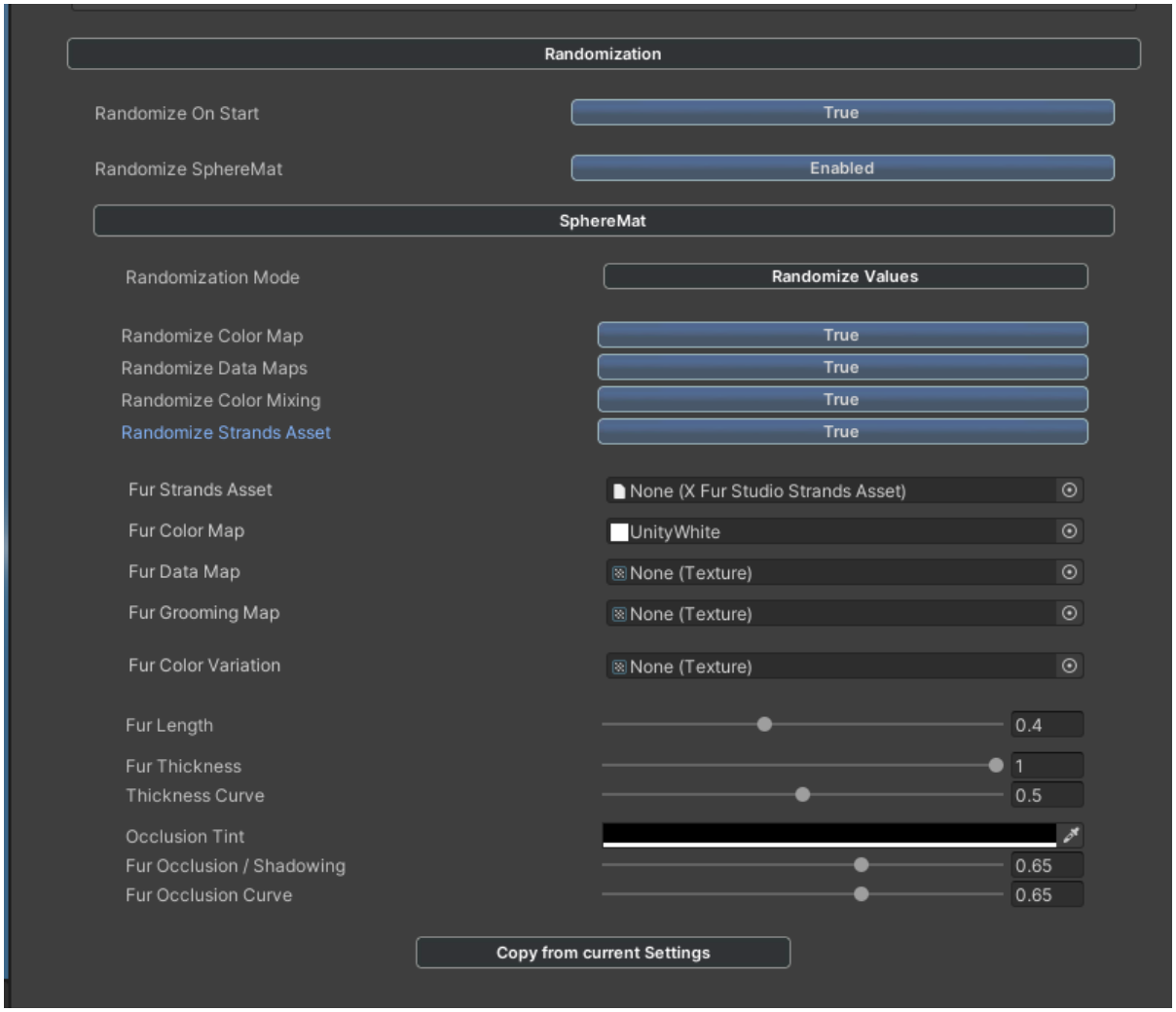
Decals can be swapped at runtime since they are added in a non-destructive way. They can have different offset and tiling values for each decal, allowing great flexibility when customizing your characters

Randomization Module

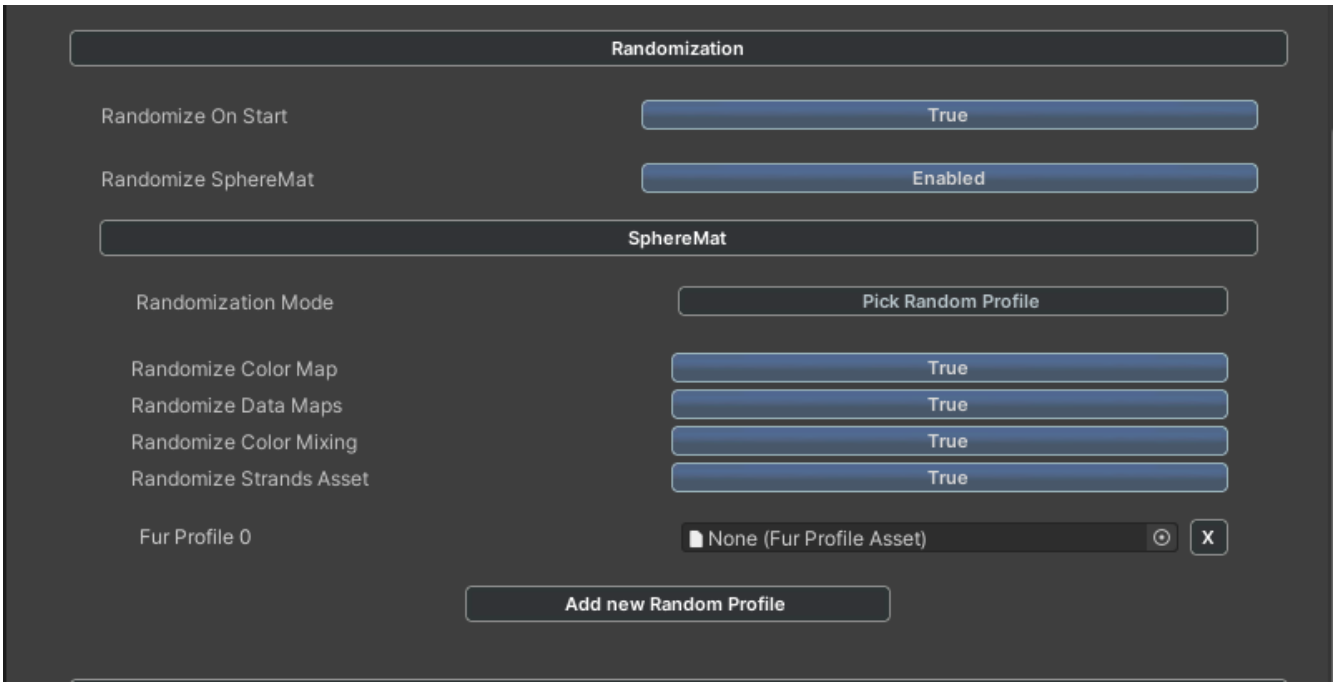
The randomization module allows you to generate endless variations for your characters by manipulating all fur parameters at runtime within certain specific rules.

The randomization module can be run in three different modes: Randomized values, Pick Random Profile, and Randomize Values and Profiles.

The randomization can be triggered automatically upon Start or manually through code. When using the Randomize Values mode you can provide a series of alternative values for most fur parameters on a per-material basis. When active, the Randomization module will generate a new variant by selecting random values for each parameter that are located between their original values and the alternative values provided by you.

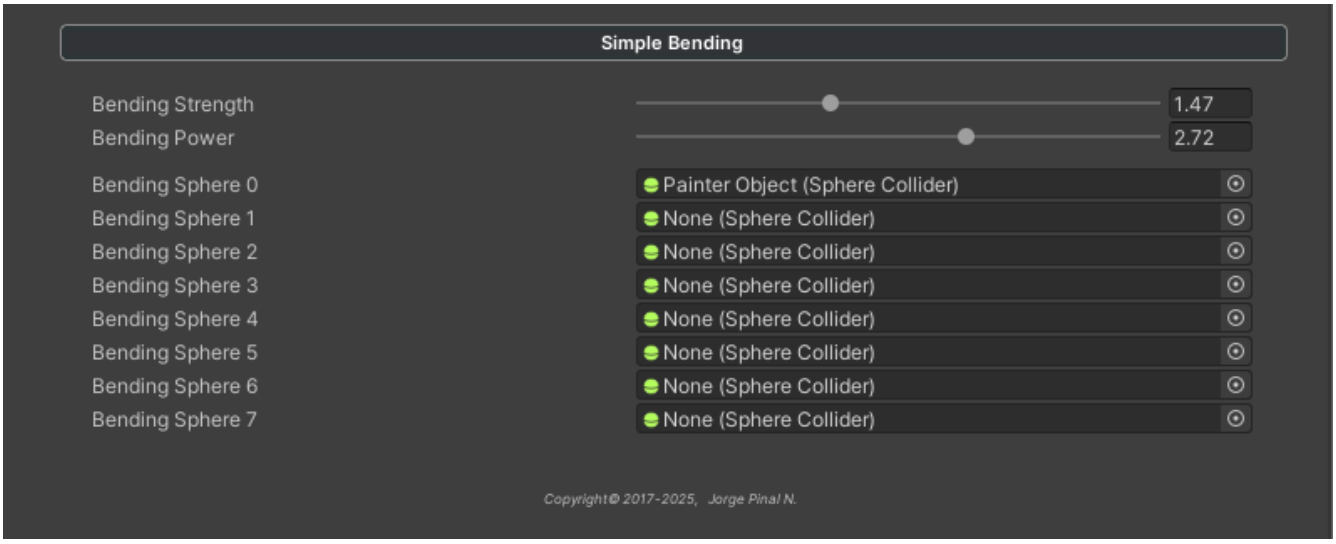


When using the Pick Random Profile mode, a list of Fur profiles is provided to the module for it to pick one at random. If you select the Randomize Values and Profiles mode each parameter will be randomized between the values of the original parameters and those of a random profile from the list.

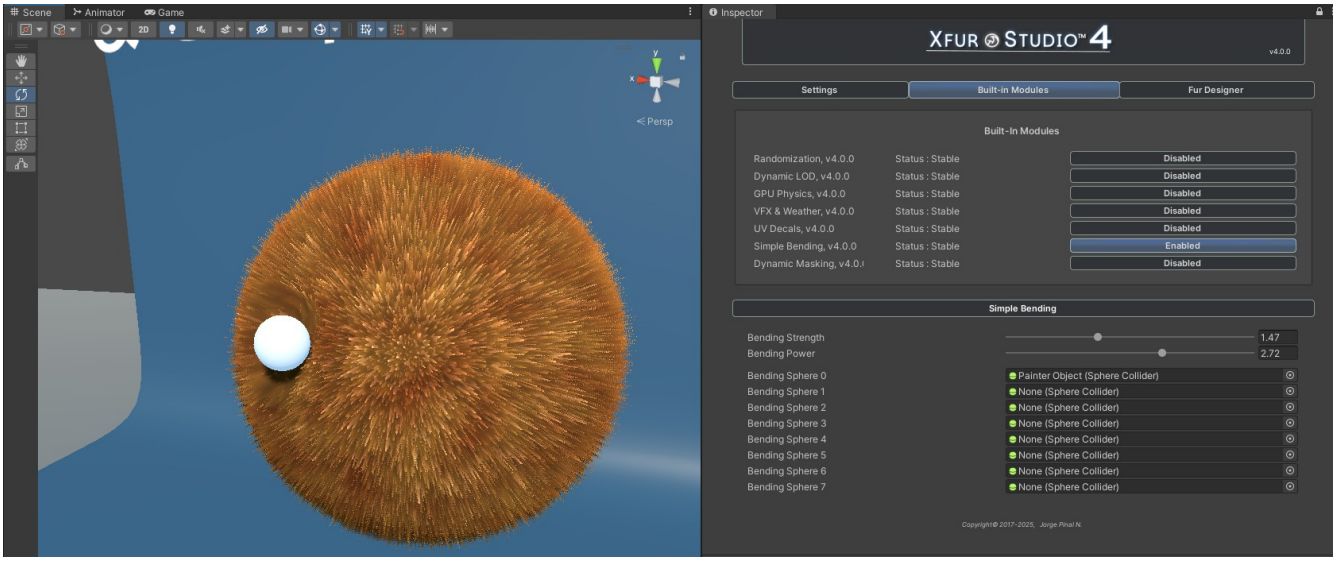


For any randomization mode, you can select whether to randomize only the general fur parameters or also the different textures and even the Strands Asset, which can allow you to have an almost infinite number of variations.

Simple Bending Module

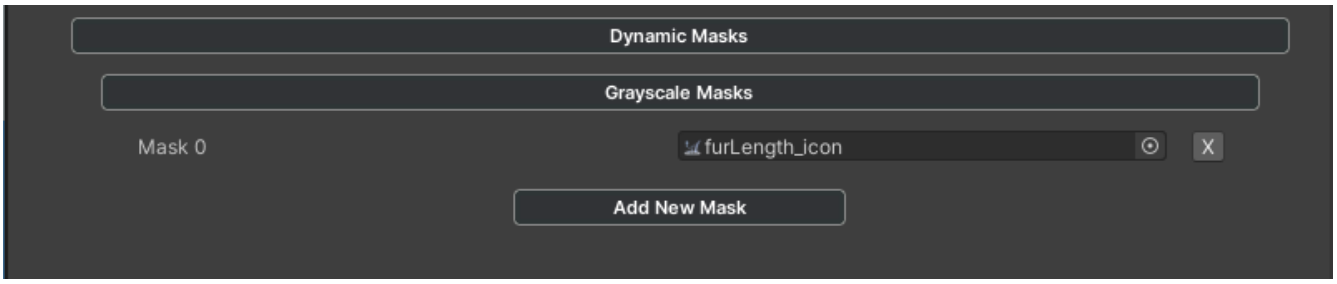


The simple bending module allows you to specify up to 8 sphere colliders for each XFur Studio™ Instance and let them bend and push the fur away. This can be used to simulate simple interactions and forces being applied to the fur.

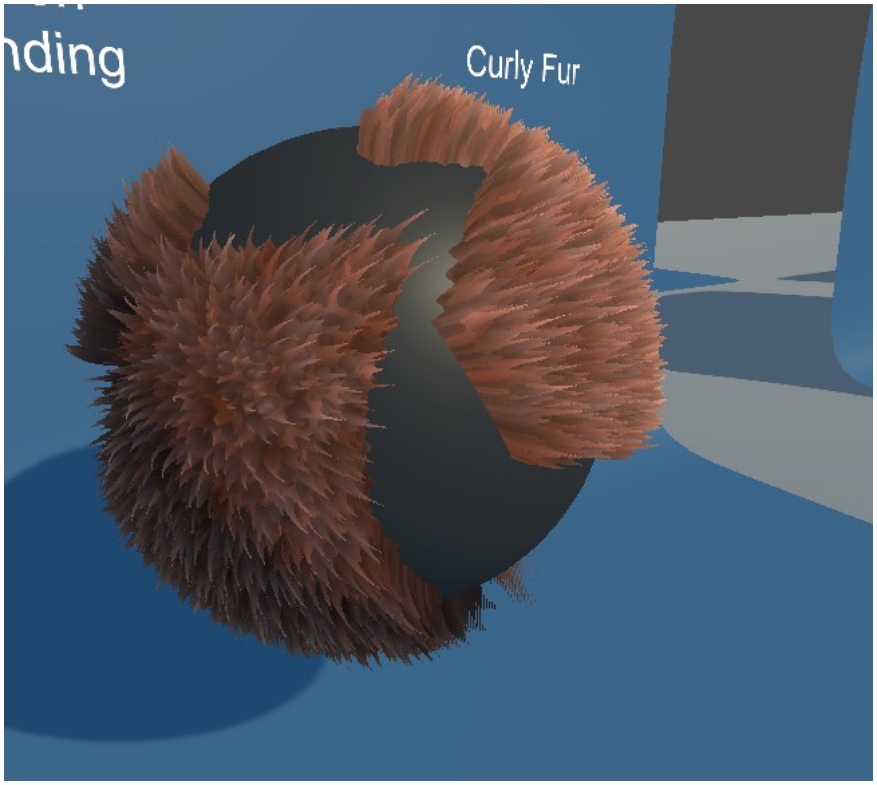


A simple demonstration of a sphere bending the fur of another sphere

Dynamic Masking Module



The Dynamic Masking module allows you to specify any number of grayscales as additional masks to remove fur from your model dynamically. This is useful for characters that, for example, need to wear different outfits that may block the fur from rendering and will need to be swapped at runtime.



Dynamic masking in action

Additional Rendering Features

Important Notice: These additional Rendering Features are not available in XFur Studio™ 4 – Core Edition. To use them you will need to upgrade to the URP, Personal or Team Editions.

Rendering Motion Vectors

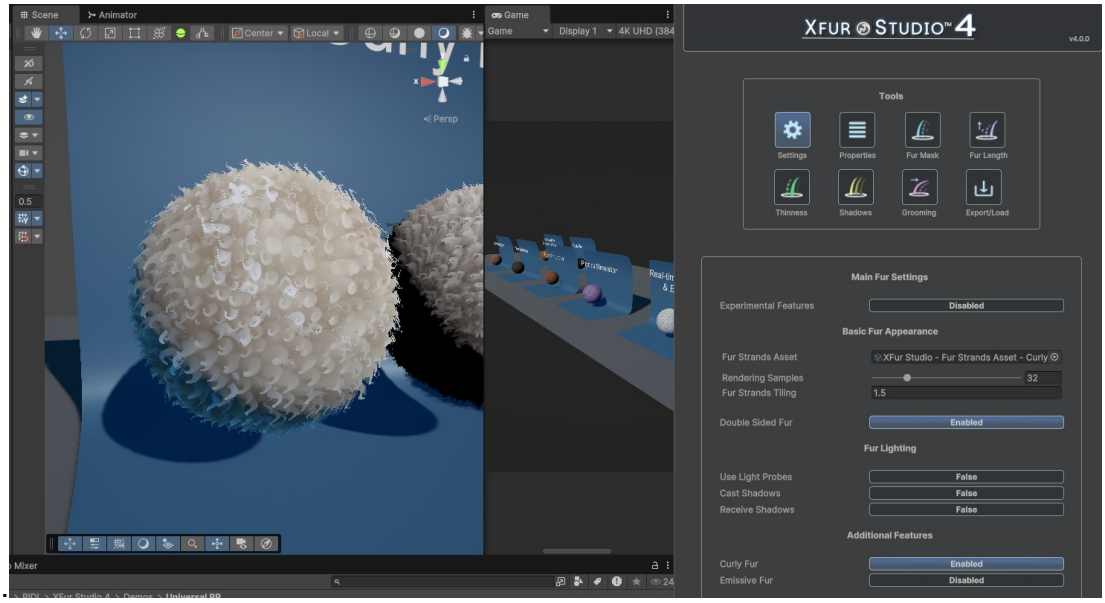
XFur Studio™ 4 is capable of rendering accurate motion vectors for the fur in the Universal Rendering Pipeline (Unity 6+) and the High Definition Rendering Pipeline (Unity 2021+). This feature allows full compatibility with Post Process FX such as temporal anti-aliasing, motion blur and the new Spatial-Temporal Post-Processing upscaler without any additional artifacts.

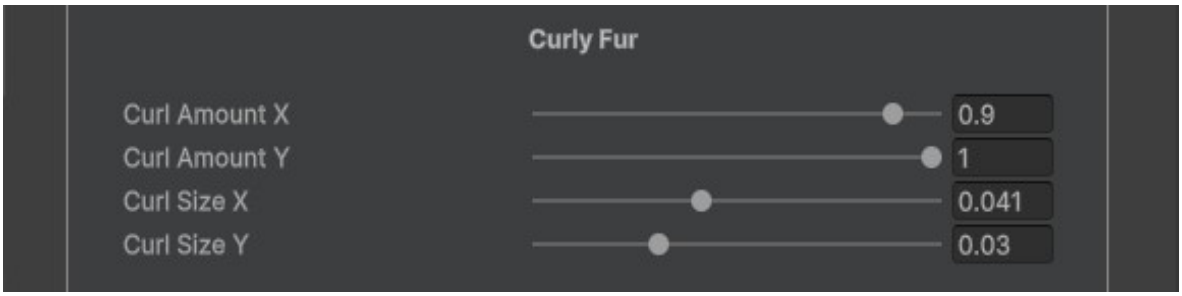
A comparison between no-motion vectors with Temporal anti-aliasing enabled (left) and with accurate motion-vectors being rendered (right).

Curly Fur

XFur Studio™ 4 allows you to easily simulate curly fur by twisting the fur strands through UV manipulation.

To enable the Curly fur parameters simply enable Curly fur in the Settings tab of XFur Studio Designer, and then head to Fur Properties to adjust it.

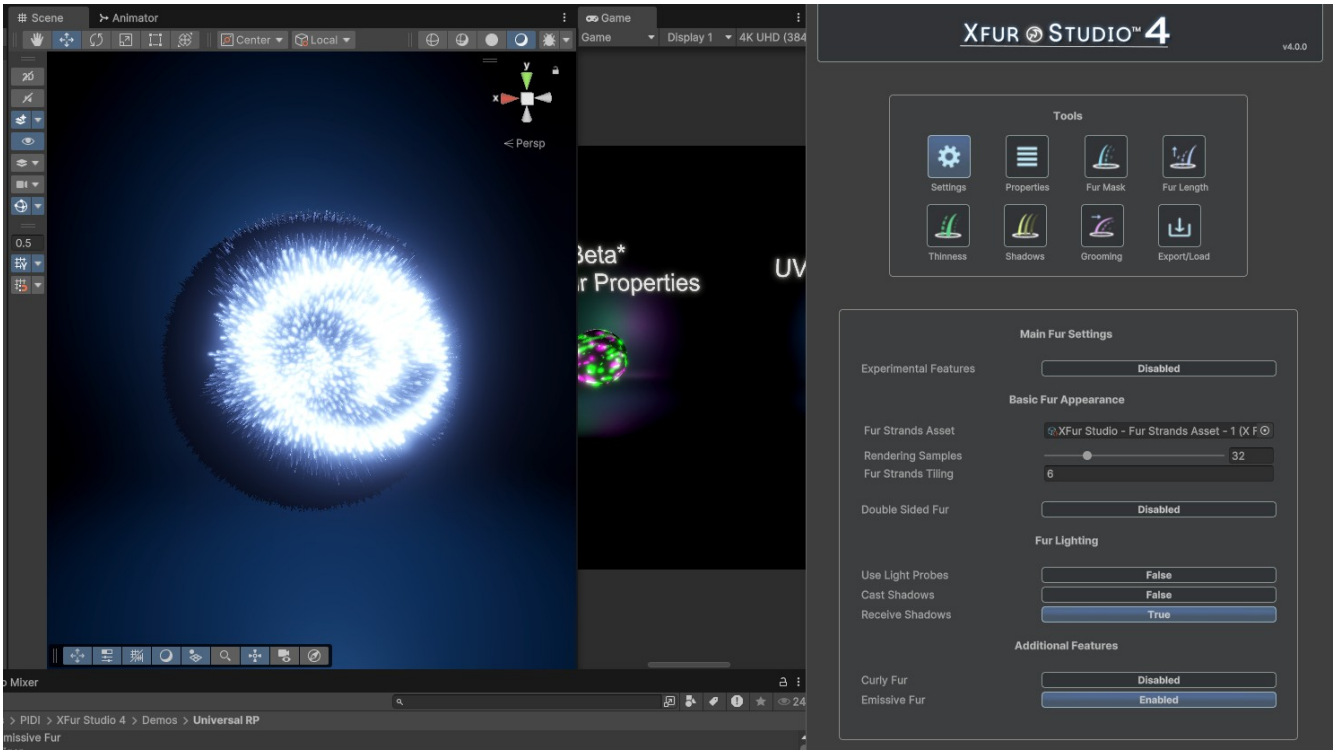




The available parameters control the number of waves / curls to be generated on the X and Y axes as well as their size. By playing with these parameters, you can create many different kinds of curly fur, from thin and well defined “spring-like” curls to wool-like materials, etc. Curly fur is compatible with all XFur modules including physics and weather FX.

Emissive Fur

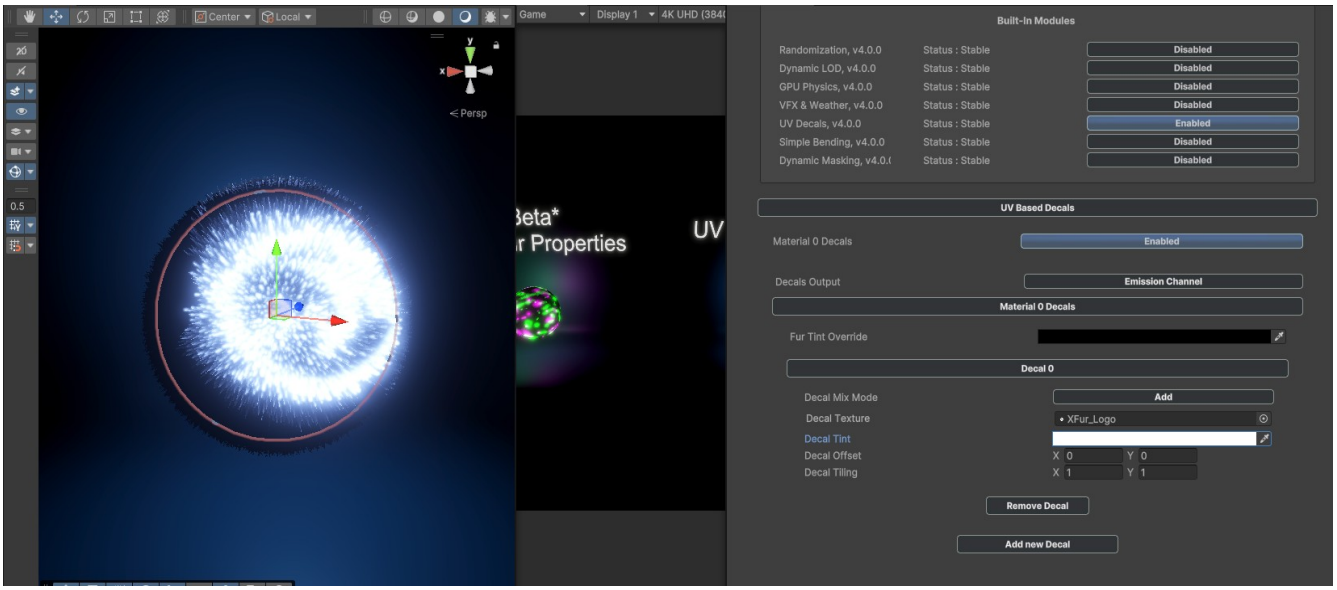
Fur can also emit light, which is especially useful for magical creatures. You can enable emissive fur in the Settings tab of XFur Studio™ Designer.



Activating Emissive Fur through the Fur Designer / Settings tool

Once enabled, you can select an emission map and an emissive color to adjust the strength and appearance of the effect.

Additionally, the decals module can also output decals to the Emission channel when Emissive fur is enabled, giving even more control over how the effects play over the surface of your models.

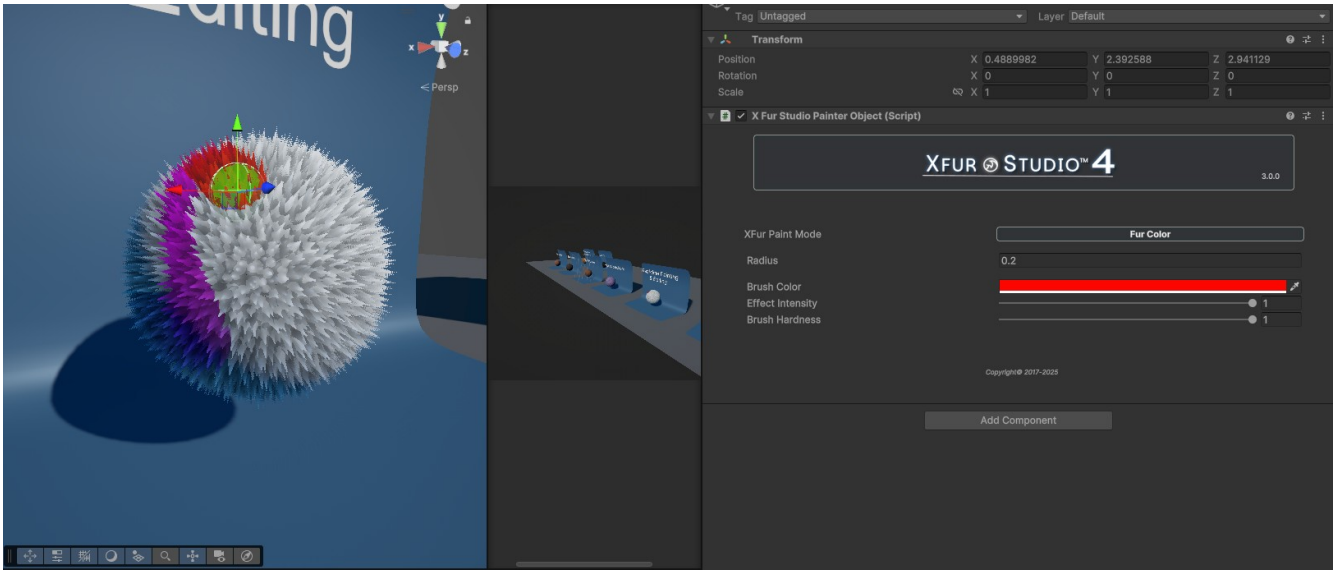


Emissive decals in action

XFur Studio Painter Objects

Painter Objects are extremely useful “Virtual Brushes” that allow you to modify the appearance of the fur at runtime in a similar fashion to XFur Studio™ Designer, but they work at runtime and can easily be attached to other objects to be implemented within different gameplay mechanics.

Shaving, grooming or coloring the fur as part of your game is extremely easy with the XFur Studio™ Painter Objects.



Depending on the Paint Mode selected the UI of the Painter Object will automatically adjust itself to expose the right parameters to use, either a Brush color, effects intensity, whether to invert the painting mode (when painting the Fur Mask for example, it allows you to either add or shave away fur) etc. To add an XFur Studio™ Painter Object to your scene, simply create an empty game object and add the “XFurStudioPainterObject” component to it.

Troubleshooting

1- The fur looks strange / the strands are not projected correctly / fur is missing

Please ensure that your model has either no secondary UV channel or that it has been unwrapped properly. XFur Studio™ makes use of the second UV channel to project the fur strands, and in most models this is a copy of the main UV channel or it is properly unwrapped for lightmaps usage. You must either unwrap the model correctly or, if you do not have access to the source mesh and / or a 3D modeling software, enable the "Generate Light Map UVs" option when importing the model.

2- Fur looks too short / too long

This error is caused in most cases by the source mesh having an incorrect scale upon being imported. Select your Mesh or Skinned Mesh Renderer and check its Transform component. If the scale is not 1,1,1 then it will produce fur that is either too long or too short.

3- Painting does not work / brush can't detect the model

This means that the automatic mesh collider being generated for your model has the wrong size and / or rotation. This can happen if the mesh itself was not properly rotated and scaled upon being imported. Some Blender models have a 90 degrees rotation on the X axis and can have a scale of 100,100,100 due to the way Blender handles FBX files by default. Ensure that your model is uniformly scaled and that its location / rotation are set to zero by checking the Transform component in the Mesh / Skinned Mesh Renderer component, and fix it on your preferred 3D modeling app if this is not the case. If the model was provided by a third-party, contact them to have this issue solved.

4- Grooming / symmetry not working as expected

For best results when using symmetry mode while painting with XFur Studio™ Designer, ensure that your model has no rotation applied to it and that its scale is a uniform 1,1,1 scale. If the problem persists, attempt to paint it while the model is placed at coordinates 0,0,0 in the scene and with a rotation of 0,0,0 as well.

5- Temporal anti-aliasing makes the fur look blurry / unstable / with ghosting

By default, XFur Studio™ does not render motion vectors which are required for anti-aliasing and other motion dependent effects to work. In the Universal Rendering Pipeline (Unity 6+) and the High Definition Rendering Pipeline (Unity 2021+) you can enable Motion Vectors from the settings of your XFur Studio Instance component. Motion Vectors are not supported in the Legacy pipeline nor in Unity projects using URP and a Unity version prior to Unity 6. For this cases, we recommend using Subpixel Morphological Anti-aliasing instead.

6- A lower end device runs slow / heats when looking at the fur or fur heavy scenes

Fur rendering is a heavy effect and many Unity apps can overheat some devices when using the CPU in an efficient way (for example, when Burst is enabled). If this is the case, there are a few solutions you can attempt to improve performance and reduce the load on your device.

- **Lower the base resolution and upscale with your preferred upscaling method**
- **Lower the amount of samples used**
- **Lower the framerate of the application using V-Sync or by limiting the framerate manually**
- **Reduce the amount of screen-space covered by the fur to reduce overdrawing**

XFur Studio™ API Reference

To start using XFur Studio™ 4 with your own custom code, always remember to add the XFur Studio 4 assembly definition to your own assembly definition files.

There are a few different namespaces used within XFur Studio™ 4. For each class described in this section we will also include the full namespace required to use them and reference them.

Every class, property and method is heavily commented and documented in-line (through <summary> blocks) to assist you when using XFur Studio™ through your preferred IDE.

XFurStudioInstance

Class in XFurStudio.Core. Inherits from MonoBehaviour.

Properties

Property	Type	Description
Settings	**XFurStudioInstanceSettings**	The settings for this XFur Studio Instance.
RenderingResources	**XFurRenderingResources**	The internal rendering resources of this instance, like its material, shaders and current rendering pipeline
FurDataProfiles	**FurProfileData[]**	The internal fur profile data sets to be assigned to each material
CustomModules	**XFurStudioModuleAsset[]**	The Custom Modules array assigned to this instance
IsSkinnedMesh	**bool**	Whether the current mesh displaying the fur is a skinned mesh or not
CurrentMesh	**Mesh**	The current mesh used to display the fur
CurrentFurRenderer	**XFurMeshRendererData**	The renderer currently in use to display the fur
RandomizationModule	**XFurRandomizationModule**	The randomization module assigned to this instance
LODModule	**XFurLodModule**	The Dynamic LOD module assigned to this instance
PhysicsModule	**XFurPhysicsModule**	The Physics Module assigned to this instance
VFXModule	**XFurVFXModule**	The VFX Module assigned to this instance
DecalsModule	**XFurDecalsModule**	The UV Decals module assigned to this instance
SimpleBendingModule	**XFurSimpleBendingModule**	The Simple Bending module assigned to this instance
DynamicMasksModule	**XFurDynamicMaskingModule**	The Dynamic Masking module assigned to this instance
WeatherManager	**XFurWeatherManager**	The global XFur Wind and Weather manager affecting this fur simulation

Methods

GetFurData

void GetFurData(int index, out FurProfileData furProfileData)

Parameter	Description
int index	The index of the material for which we will retrieve all fur properties
FurProfileData furProfileData	The output FurProfileData struct the requested properties

It retrieves all fur properties set for a given material index on this XFur Studio Instance as a FurProfileData struct. Result is passed by value and therefore the original properties will remain unchanged.

ReplaceFurData

void ReplaceFurData(int index, FurProfileData data)

void ReplaceFurData(int index, FurProfileAsset asset)

Parameter	Description
int index	The index of the material that will receive the new properties
FurProfileData data	The FurProfileData struct with the properties we want to set
FurProfileAsset asset	Alternatively, a FurProfileAsset can be used to set the data of the material

It replaces the fur profile properties of a given material with the properties contained in the given FurProfileData or the internal profile of a FurProfileAsset

SetFurData

void SetFurData(int index, FurProfileData data, bool replaceColorMap, bool replaceDataMaps, bool replaceColorMix, bool replaceStrandsMap)

void SetFurData(int index, FurProfileAsset asset, bool replaceColorMap, bool replaceDataMaps, bool replaceColorMix, bool replaceStrandsMap)

Parameter	Description
int index	The index of the material that will receive the new properties
FurProfileAsset data	The FurProfileData struct with the properties we want to set
FurProfileAsset asset	Alternatively, a FurProfileAsset can be used to set the data of the material
bool replaceColorMap	Optional. Whether to replace the color map and properties or not
bool replaceDataMaps	Optional. Whether to replace the data and grooming or not
bool replaceColorMix	Optional. Whether to replace the color variation properties or not
bool replaceStrandsMap	Optional. Whether to replace the strands map or not

Sets the fur profile data for a given index, with fine control over which data gets replaced and which remains untouched.

UpdateFurMaterials

void UpdateFurMaterials(bool forceUpdate)

Parameter	Description
bool forceUpdate	Optional. Forces an update regardless of the auto-update settings.

Applies all changes to the fur properties across all materials and forcefully updates any modules that work on the render loop

XFurStudioInstanceSettings

Class in XFurStudio.Core

Properties

Property	Type	Description
useVertexBuffer	**bool**	Enables the use of vertex buffers to synchronize the fur with the animated mesh
renderMotionVectors	**bool**	Enables the use of dual vertex buffers, to render motion vectors
useNormalmap	**bool**	Enables the use of normalmaps with the fur
useCustomModules	**bool**	Enables the use of custom modules in the related XFur Studio Instance
useBakedVertexData	**bool**	Whether the XFur Studio Instance component will use baked data instead of maps
urpAdvancedLighting	**bool**	Enables the use of advanced URP Lighting (anisotropic highlights)
autoUpdateMaterials	**bool**	Whether the changes done to the fur will be applied automatically
timeBetweenUpdates	**float**	The time between each automatic update
autoCompensateForScale	**bool**	Allows fur properties to be slightly adjusted to account for a model's scale
useLossyScale	**bool**	Uses the actual world-based scale of the model rather than the local one
experimentalFeatures	**bool**	Enables experimental features in this XFur Studio Instance
version	**Vector3Int**	The current version of this XFur Studio Instance. Used for automatic updates.

XFurRenderingResources

Struct in XFurStudio.Core

Properties

Property	Type	Description
currentRenderingPipeline	**XFurRenderingPipeline**	Simple enum detailing which pipeline is active in the project **XFurRenderingPipeline{ LegacyRP, UniversalRP, HDRP }**
legacyShellsFurShader	**Shader**	The internal fur shader used for the legacy pipeline
urpShellsFurShader	**Shader**	The internal fur shader used for the universal rendering pipeline
hdrpShellsFurShader	**Shader**	The internal fur shader used for the HD rendering pipeline
internalFurMaterial	**Shader**	The current fur material, auto-configured and ready to use
internalFurMaterialDoubleSided	**Shader**	The current fur material, set up as a double-sided material
emptyNormalmap	**Shader**	A default, empty normal map with a 1x1 resolution

FurProfileData

Struct in XFurStudio.Core

Properties

Property	Type	Description
version	**XFurRenderingPipeline**	The version of this FurProfileData. Used for self-updating
useProbes	**bool**	Enables compatibility with Unity Light Probes
castShadows	**bool**	Whether the fur will cast shadows or not
receiveShadows	**bool**	Whether the fur will receive shadows or not
rimLightingTint	**Color**	The tint of the rim lighting effect applied to the fur
rimLightingStrength	**float**	Multiplies the strength of the rim lighting effect to boost it or dim it
rimLightingPower	**float**	Controls the thickness of the rim lighting effect
renderingSamples	**float**	How many rendering samples (shells) to use to render the fur
mainTint	**Color**	The main tint to be applied to the fur
colorMap	**Texture**	The main color texture for the fur (the albedo map)
baseUVTiling	**Vector4**	The base tiling and offset to apply to the color and normal maps
furStrandsAsset	**FurStrandsAsset**	The FurStrandsAsset that describes the appearance of the fur
furStrandsTexture	**Texture**	Optionally, an external fur strands map
furStrandsTiling	**float**	The tiling to apply to the fur strands
doubleSided	**bool**	Whether the fur will be rendered with a double sided material or not
furLength	**float**	The length of the fur
furThickness	**float**	The fur’s thickness
furThicknessCurve	**float**	The thickness variation from the root to the tip of a fur strand
useCurlyFur	**bool**	Whether to use the curly fur properties or not
curlyFurParameters	**Vector4**	Curliness (X, Y) and size of the curl twists (Z, W)

Property	Type	Description
emissiveFur	**bool**	Whether emissive fur will be used or not
emissiveTint	**Color (HDR)**	The HDR color used for the emission channel
useLegacyColorVariation	**bool**	Whether color variation will use the legacy method
legacyColorVariationMap	**Texture**	A texture that defines color variation through its color channels
furNoiseShadingTiling	**float**	The tiling of the procedural noise variation pass
noiseShadingTint0	**Color**	First tint for color variation
noiseShadingTint1	**Color**	Second tint for color variation
noiseShadingTint2	**Color**	Third tint for color variation
noiseShadingTint3	**Color**	Fourth tint for color variation
mainFurStrandBoost	**float**	A boost / multiplier for the color of the main fur strands
secondaryFurStrandBoost	**float**	A boost / multiplier for the color of the secondary fur strands
useVertexData	**bool**	Whether to use the Baked Vertex Data mode or not
furDataMap	**Texture**	The fur data map, used for fur styling
furGroomingMap	**Texture**	The fur grooming map, used for fur styling
groomStrength	**float**	The strength of the grooming effect over the fur
windStrengthMultiplier	**float**	Multiplies the global wind strength for this particular instance
normalMap	**Texture**	The normal map to apply to the fur
selfOcclusionStrength	**float**	The strength of the self-shadowing effect
selfOcclusionCurve	**float**	The variation of the shadowing effect over the strand of fur
selfOcclusionTint	**Color**	The tint for the self-shadowing effect
roughness	**float**	The roughness of the fur. Less roughness makes it shinier
metallic	**float**	The metallness of the fur.
specularTint	**Color**	The tint applied to the specularity of the fur material

XFurStudioPainterObject

Class in XFurStudio.Utilities. Inherited from MonoBehaviour

Properties

Property	Type	Description
PaintDataMode	**PaintDataMode**	Enum defining the kind of brush used by this Painter Object
InvertBrush	**bool**	Whether the brush's effect should be inverted or not
BrushOpacity	**float**	The opacity of the brush
BrushHardness	**float**	The hardness of the brush (controls the strength from the center to the outer portion of the virtual brush sphere)
BrushColor	**Color**	The brush's color. For most painting requests, it is ignored.
Radius	**float**	The radius of the virtual brush sphere

XFurStudioAPI

Class in XFurStudio.Core. Used for dynamic painting and styling at runtime and in the Editor.

Methods

LoadPaintResources

void LoadPaintResources( )

Loads all the necessary resources to paint and edit the fur at runtime, including all color mixing and 3D painting shaders.

Groom

void Groom(XFurStudioInstance target, int matIndex, Vector3 brushCenter, Vector3 brushNormal, float brushSize, float brushOpacity, float brushHardness, Vector3 groomDirection, bool invert = false)

Parameter	Description
XFurStudioInstance target	The XFur Studio Instance we will be grooming
int matIndex	The fur material that we will be applying the grooming effect to
Vector3 brushCenter	The center in world space coordinates of the virtual 3D brush
Vector3 brushNormal	The direction in which the brush is “looking” or being applied
float brushSize	The size of the brush, its radius as a 3D sphere
float brushOpacity	The opacity of the brush. Affects the strength of the effect
float brushHardness	The hardness of the brush. Makes the strength of the effect more or less uniform when going from the center to the outer portion of the brush
Vector3 groomDirection	The direction in which the brush is moving, in world space coordinates
bool invert	Whether the effect should be inverted (grooming should be removed instead of being applied over the fur)

Allows dynamic grooming of the fur of an XFur Studio instance through code at runtime. For implementations of this function you can look at the XFur Studio™ Designer code or, for a more runtime-oriented application, to the XFur Painter Object source code.

Paint

void Paint(XFurStudioInstance target, PaintDataMode painterMode, int matIndex, Vector3 brushCenter, Vector3 brushNormal, float brushSize, float brushOpacity, float brushHardness, Color brushColor)

Parameter	Description
XFurStudioInstance target	The XFur Studio Instance we will be grooming
PaintDataMode painterMode	Enum indicating the kind of value that will be painted into the fur
int matIndex	The fur material that we will be applying the grooming effect to
Vector3 brushCenter	The center in world space coordinates of the virtual 3D brush
Vector3 brushNormal	The direction in which the brush is “looking” or being applied
float brushSize	The size of the brush, its radius as a 3D sphere
float brushOpacity	The opacity of the brush. Affects the strength of the effect
float brushHardness	The hardness of the brush. Makes the strength of the effect more or less uniform when going from the center to the outer portion of the brush
Color brushColor	The color of the brush we will be painting with

Allows dynamic data painting for the different fur property maps of an XFur Instance using a virtual world space based brush. For implementations of this function you can look at the XFur Studio™ Designer code or, for a more runtime-oriented application, to the XFur Painter Object source code.

PaintDataMode

Enum in XFurStudio.Core, defined within the XFurStudioAPI class.

Declaration: PaintDataMode { FurMask, FurLength, FurThickness, FurOcclusion, FurColor, SnowFX, BloodFX, WaterFX, FurGrooming }